

1. Introducción

1.1 Objetivo

Synthea es un motor de simulación clínica de código abierto diseñado para generar pacientes sintéticos y sus correspondientes historias clínicas longitudinales. A diferencia de los generadores tradicionales de datos, Synthea no crea registros independientes, sino que simula la evolución temporal completa de individuos virtuales siguiendo reglas clínicas, probabilidades epidemiológicas y guías médicas.

El objetivo principal de Synthea es proporcionar conjuntos de datos clínicos sintéticos que puedan utilizarse para investigación, validación de software sanitario, interoperabilidad (FHIR, CDA, CSV), inteligencia artificial y desarrollo de sistemas de información clínica sin comprometer la privacidad de pacientes reales.

1.2 Filosofía del sistema

La idea fundamental de Synthea consiste en simular vidas completas en lugar de generar registros clínicos aislados.

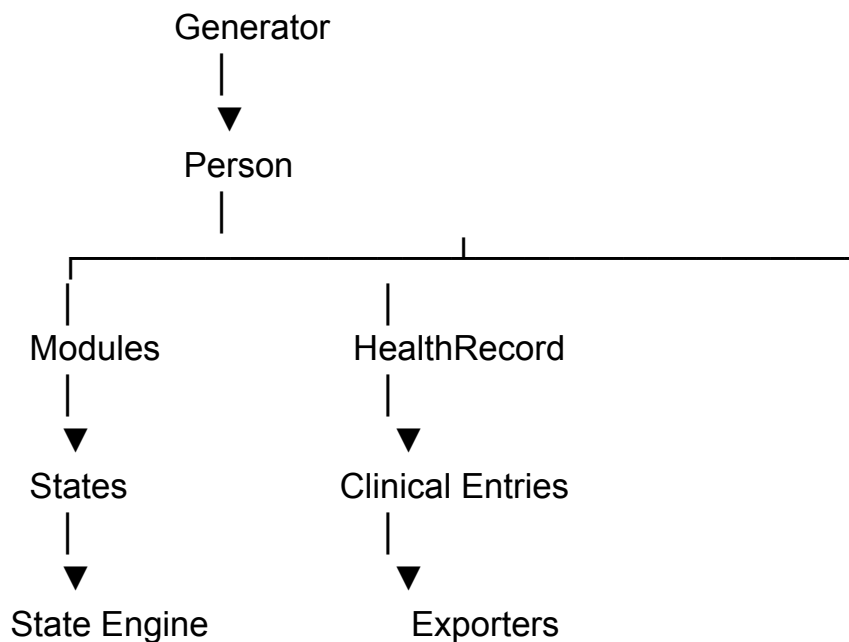
Cada paciente es tratado como un individuo cuya historia evoluciona desde el nacimiento hasta el fallecimiento. Durante esta simulación aparecen enfermedades, procedimientos, visitas médicas, medicación y eventos clínicos coherentes entre sí.

Como consecuencia, la historia clínica generada constituye el resultado natural de la simulación y no un conjunto de datos generados aleatoriamente.

2. Arquitectura general

2.1 Visión de alto nivel

Synthea sigue una arquitectura claramente separada en responsabilidades.



Cada componente posee una única responsabilidad, permitiendo mantener una arquitectura modular y fácilmente extensible.

3. Componentes principales

3.1 Generator

Responsabilidad

El Generator constituye el punto de entrada del motor de simulación.

Su función consiste en crear la población sintética, inicializar los pacientes y ejecutar la simulación completa para cada individuo.

No contiene lógica clínica.

Simplemente coordina el proceso de simulación.

Responsabilidades

- Inicialización de pacientes.
 - Gestión del tiempo de simulación.
 - Ejecución de módulos.
 - Coordinación general del motor.
-

3.2 Person

Responsabilidad

La clase Person representa al paciente durante toda su vida.

Todos los módulos modifican esta misma instancia.

Person no implementa reglas clínicas.

Su función consiste únicamente en almacenar el estado completo del paciente.

Información almacenada

- Datos demográficos.
- Edad.
- Sexo.
- Atributos.
- Enfermedades activas.
- Medicación.
- Procedimientos.
- Observaciones.
- Historia clínica.

El paciente constituye el eje central de toda la simulación.

3.3 Module

Responsabilidad

Un módulo representa un proceso clínico completo.

No representa únicamente una enfermedad.

Puede modelar cualquier proceso asistencial:

- Hipertensión.
- Diabetes.
- Embarazo.
- Vacunación.
- Urgencias.
- Cuidados paliativos.

Cada módulo se implementa mediante un archivo JSON independiente.

Los módulos contienen exclusivamente lógica declarativa.

El motor es quien interpreta dicha lógica.

3.4 States

Responsabilidad

Los estados constituyen las unidades fundamentales de ejecución dentro de un módulo.

Cada estado posee una única responsabilidad y modifica el paciente de una forma concreta.

Ejemplos:

- Initial
- Delay
- Encounter
- ConditionOnset
- MedicationOrder
- Procedure
- Observation
- ConditionEnd
- Terminal

Los estados son ejecutados secuencialmente por el motor siguiendo las transiciones definidas en el módulo.

3.5 HealthRecord

Responsabilidad

HealthRecord representa la historia clínica completa del paciente.

No contiene lógica clínica.

Su única función consiste en almacenar los eventos producidos durante la simulación.

Entre ellos:

- Diagnósticos.
- Consultas.
- Procedimientos.
- Medicación.
- Observaciones.
- Hospitalizaciones.

HealthRecord constituye posteriormente la fuente de información utilizada por los exportadores.

3.6 Exporters

Responsabilidad

Los exportadores transforman la historia clínica generada en distintos formatos.

No modifican la simulación.

No crean información adicional.

Simplemente serializan la información contenida en HealthRecord.

Principales formatos:

- FHIR

- CSV
 - CDA
 - JSON
-

4. Filosofía de los módulos

Los módulos representan protocolos clínicos completos.

Su implementación es completamente declarativa mediante JSON.

Cada módulo está formado por un conjunto de estados conectados mediante transiciones.

El motor interpreta dichos estados sin necesidad de modificar el código Java.

Esta separación permite desacoplar la lógica clínica del motor de simulación.

5. Máquina de estados

Synthea implementa una arquitectura basada en máquinas de estados.

Cada módulo puede representarse como un grafo dirigido.

Initial



Evaluate Risk



Encounter



ConditionOnset

↓

MedicationOrder

↓

Observation

↓

Terminal

Cada transición puede depender de:

- Condiciones clínicas.
- Edad.
- Sexo.
- Probabilidades.
- Factores de riesgo.
- Resultados de estados anteriores.

Esta arquitectura permite representar protocolos clínicos complejos manteniendo una gran flexibilidad.

6. Flujo completo de una enfermedad

La aparición de una enfermedad sigue el siguiente recorrido arquitectónico.

JSON Module

↓

ConditionOnset State

↓

State.java

↓

diagnose()



Person



HealthRecord.conditionStart()



Encounter



Condition Entry



FHIR Exporter

Este flujo resume la interacción entre los distintos componentes del sistema.

7. Principios de diseño

Durante el análisis del código fuente se identifican diversos principios de diseño.

Separación de responsabilidades

Cada clase implementa una única responsabilidad claramente definida.

Ejemplos:

- Generator coordina.
- Person almacena.

- State ejecuta.
 - HealthRecord registra.
 - Exporter exporta.
-

Separación entre motor y conocimiento clínico

La lógica clínica reside en los módulos JSON.

El motor únicamente interpreta dichos módulos.

Esto permite modificar protocolos clínicos sin alterar la implementación del motor.

Arquitectura orientada a eventos

Cada modificación realizada sobre el paciente genera un evento clínico.

Estos eventos son almacenados cronológicamente en la historia clínica.

La historia clínica constituye el resultado final de la simulación.

Arquitectura desacoplada

Los módulos desconocen la implementación interna del motor.

El motor desconoce el contenido clínico específico de cada módulo.

Ambos componentes únicamente se comunican mediante estados estandarizados.

8. Flujo de ejecución completo

Inicio de simulación



Generator crea Person



Carga de módulos JSON



Inicialización del primer estado



Ejecución de estados



Actualización del Person



Registro en HealthRecord



Avance temporal



Nuevos módulos



Finalización de la vida del paciente



Exportación FHIR / CSV / CDA

9. Decisiones de arquitectura relevantes

Durante el estudio del código original deben evaluarse especialmente las siguientes decisiones de diseño:

- Separación entre motor y módulos clínicos.
- Representación declarativa mediante JSON.
- Uso de una máquina de estados para modelar protocolos clínicos.
- Existencia de un objeto central (Person).
- Separación entre estado del paciente e historia clínica.
- Exportación desacoplada respecto al motor.
- Independencia entre la lógica clínica y los formatos de salida.

Estas decisiones servirán posteriormente como base para comparar la implementación original con la adaptación en Python.

10. Criterios para comparar la versión Python

El análisis de la adaptación en Python debería responder, al menos, a las siguientes cuestiones:

1. ¿Se mantiene la arquitectura basada en módulos?
2. ¿Los módulos siguen representándose mediante JSON o han sido reimplementados en Python?
3. ¿Existe un equivalente funcional de **Person**?
4. ¿Cómo se representa la historia clínica?
5. ¿Se mantiene la separación entre motor y lógica clínica?
6. ¿Cómo se implementan los estados y las transiciones?
7. ¿Qué componentes del motor original se han conservado?
8. ¿Qué componentes se han rediseñado?
9. ¿Qué modificaciones se han introducido específicamente para adaptar la simulación al sistema sanitario español?
10. ¿La adaptación mantiene la filosofía arquitectónica de Synthea o introduce un nuevo paradigma de simulación?

CAPÍTULO 1

Introducción a Synthea

1.1 ¿Qué es Synthea?

Synthea es un motor de simulación clínica de código abierto desarrollado originalmente por The MITRE Corporation cuyo objetivo es generar pacientes sintéticos y sus correspondientes historias clínicas longitudinales.

A diferencia de los generadores tradicionales de datos, Synthea no crea registros independientes o aleatorios, sino que **simula la evolución completa de la vida de un individuo**, desde su nacimiento hasta su fallecimiento, reproduciendo de forma coherente los eventos clínicos que podrían ocurrir durante ese periodo.

El resultado final es una historia clínica sintética con un elevado grado de realismo clínico y estructural, apta para investigación biomédica, validación de software sanitario, desarrollo de algoritmos de inteligencia artificial, pruebas de interoperabilidad y formación.

1.2 El problema que intenta resolver

El desarrollo de herramientas basadas en datos clínicos presenta un problema fundamental: **el acceso a datos reales está limitado por cuestiones éticas, legales y de privacidad.**

Los hospitales almacenan millones de historias clínicas, pero su utilización está condicionada por normativas como el Reglamento General de Protección de Datos (RGPD) en Europa o HIPAA en Estados Unidos.

Como consecuencia, investigadores y desarrolladores encuentran enormes dificultades para acceder a conjuntos de datos suficientemente amplios para:

- desarrollar modelos predictivos;
- validar sistemas de información clínica;
- probar implementaciones FHIR;
- entrenar algoritmos de aprendizaje automático;
- realizar simulaciones epidemiológicas.

Synthea resuelve este problema **creando pacientes completamente ficticios** cuya evolución clínica mantiene coherencia médica sin corresponder a ningún individuo real.

1.3 Filosofía de Synthea

La filosofía de diseño de Synthea puede resumirse en una única idea:

Synthea no genera historias clínicas; simula vidas. La historia clínica es simplemente el registro de esa simulación.

Esta idea es probablemente la decisión de diseño más importante de todo el proyecto.

Muchos sistemas generan datos siguiendo este procedimiento:

Paciente 1

↓

Edad = 67

↓

Diabetes = Sí

↓

Hipertensión = No

Este enfoque produce registros independientes.

Synthea utiliza una filosofía completamente distinta.

Cada paciente atraviesa una evolución temporal completa.

Nacimiento

↓

Infancia

↓

Vacunación

↓

Adolescencia

↓

Factores de riesgo

↓

Enfermedades

↓

Tratamientos

↓

Consultas

↓

Hospitalizaciones

↓

Seguimiento

↓

Fallecimiento

La historia clínica no se genera directamente.

Se construye como consecuencia de todos los eventos que experimenta el paciente durante su vida.

1.4 Simulación frente a generación de datos

Esta diferencia merece una explicación específica.

Generador tradicional

Un generador convencional suele crear registros independientes.

Paciente A

Edad = 54

Hipertensión = Sí

Diabetes = No

El siguiente paciente no tiene ninguna relación con el anterior.

Los datos aparecen directamente en la base de datos.

Synthea

Synthea sigue un enfoque completamente distinto.

El paciente posee un estado interno que evoluciona continuamente.

Nacimiento

↓

Edad = 15

↓

Comienza a fumar

↓

Aumenta el riesgo cardiovascular

↓

Desarrolla hipertensión

↓

Se prescribe tratamiento

↓

Aparecen revisiones periódicas

↓

Complicaciones

↓

Seguimiento

Cada nuevo evento depende de todos los anteriores.

Esta dependencia temporal constituye una de las características fundamentales del sistema.

1.5 ¿Qué genera realmente Synthea?

Una idea equivocada muy frecuente consiste en pensar que Synthea genera archivos CSV.

Esto es falso.

Los archivos CSV representan únicamente uno de los formatos de exportación disponibles.

El verdadero producto generado por Synthea es **una simulación clínica completa**.

Durante dicha simulación se construyen objetos que representan:

- pacientes;
- diagnósticos;
- consultas;
- procedimientos;
- observaciones;
- medicaciones;
- organizaciones sanitarias;
- profesionales;
- costes;
- aseguradoras.

Cuando la simulación finaliza, toda esta información puede exportarse a distintos formatos.

Entre ellos:

- FHIR;
- CSV;
- JSON;
- CDA.

Todos estos formatos representan exactamente la misma simulación.

La diferencia reside únicamente en la forma de representar los datos.

1.6 ¿Por qué Synthea utiliza simulación?

La simulación aporta varias ventajas fundamentales.

Coherencia temporal

Las enfermedades aparecen en un momento concreto de la vida del paciente.

Los tratamientos nunca preceden al diagnóstico.

Las consultas ocurren en fechas coherentes.

Los procedimientos únicamente aparecen cuando existe una indicación clínica.

Coherencia clínica

Las decisiones siguen protocolos médicos.

Por ejemplo:

- determinadas enfermedades aumentan la probabilidad de desarrollar otras;
 - determinados tratamientos requieren revisiones periódicas;
 - determinados procedimientos generan hospitalizaciones posteriores.
-

Reproducibilidad

Toda simulación puede reproducirse utilizando la misma semilla aleatoria (*seed*).

Esto permite generar exactamente los mismos pacientes en ejecuciones posteriores, una característica fundamental para investigación científica y validación de algoritmos.

1.7 Aplicaciones de Synthea

Actualmente Synthea se utiliza en numerosos ámbitos relacionados con la informática biomédica.

Entre los más importantes destacan:

- desarrollo de sistemas de historia clínica electrónica;
 - validación de implementaciones FHIR;
 - investigación clínica;
 - inteligencia artificial aplicada a salud;
 - enseñanza de informática médica;
 - simulación poblacional;
 - evaluación de modelos epidemiológicos;
 - desarrollo de herramientas de interoperabilidad.
-

1.8 Conceptos fundamentales

Antes de estudiar la arquitectura interna es necesario introducir los conceptos principales del sistema.

Paciente

Representa al individuo cuya vida será simulada.

No constituye un simple registro, sino un objeto que evoluciona continuamente.

Evento clínico

Representa cualquier acontecimiento médico ocurrido durante la simulación.

Ejemplos:

- diagnóstico;
 - consulta;
 - procedimiento;
 - observación;
 - medicación.
-

Historia clínica

Es el conjunto cronológico de todos los eventos experimentados por un paciente durante su vida.

La historia clínica constituye el resultado de la simulación.

Nunca es el punto de partida.

Módulo

Representa un protocolo clínico completo.

Un módulo puede describir:

- una enfermedad;
 - un embarazo;
 - una vacunación;
 - una atención en urgencias;
 - un proceso asistencial completo.
-

Estado

Es la unidad mínima de ejecución del motor de simulación.

Cada estado realiza una única acción específica.

Ejemplos:

- crear un diagnóstico;
 - prescribir un medicamento;
 - registrar una consulta;
 - esperar un periodo de tiempo.
-

Motor de simulación

Es el componente encargado de interpretar los módulos y ejecutar secuencialmente sus estados, modificando el paciente conforme avanza la simulación.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales:

- **Synthea es un motor de simulación clínica, no un generador convencional de datos.**
- **Los pacientes evolucionan a lo largo del tiempo siguiendo reglas clínicas y probabilidades epidemiológicas.**

- La historia clínica constituye el resultado final de la simulación, no su punto de partida.
 - El sistema genera pacientes sintéticos completos, aptos para investigación y desarrollo de software sanitario.
 - La arquitectura se basa en la separación entre un motor de simulación genérico y módulos clínicos independientes.
-

Engineering Notes

Decisión de diseño

Simular vidas en lugar de generar registros clínicos.

Motivación

- Mantener coherencia temporal.
- Mantener coherencia clínica.
- Generar historias clínicas longitudinales.
- Facilitar la interoperabilidad.
- Evitar inconsistencias entre eventos.

Ventajas

- Pacientes realistas.
- Historias clínicas completas.
- Relaciones causales entre eventos.
- Alta reutilización de los datos generados.

Aspectos a comprobar en la adaptación Python (IRYCIS)

- ¿Se mantiene la filosofía de simulación longitudinal?
- ¿El paciente sigue siendo el eje central de la arquitectura?
- ¿La historia clínica continúa siendo una consecuencia de la simulación?
- ¿Se conserva la separación entre motor y conocimiento clínico?

CAPÍTULO 2

Filosofía de Diseño y Decisiones Arquitectónicas

2.1 Introducción

Cuando se analiza por primera vez el código fuente de Synthea resulta fácil pensar que se trata simplemente de un proyecto Java que genera pacientes sintéticos.

Sin embargo, esta interpretación es incorrecta.

Synthea no debe entenderse como una aplicación Java, sino como una plataforma de simulación clínica cuyo comportamiento viene definido por una serie de decisiones arquitectónicas cuidadosamente diseñadas.

La mayoría del conocimiento médico del sistema **no está implementado en Java**, sino en módulos declarativos independientes.

Esta separación constituye probablemente la decisión de diseño más importante de todo el proyecto.

Antes de estudiar el código es imprescindible comprender las razones que llevaron a sus desarrolladores a diseñar el sistema de esta manera.

2.2 Primera decisión arquitectónica

Separar el motor del conocimiento clínico

En muchos sistemas tradicionales la lógica clínica se implementa directamente dentro del código fuente.

Por ejemplo, un sistema convencional podría contener código similar al siguiente:

```
if(patient.getAge() > 50){  
  
    if(patient.isSmoker()){  
  
        diagnoseHypertension(patient);  
  
    }  
  
}
```

En este diseño, las reglas médicas forman parte del propio programa.

Modificar un protocolo clínico implica modificar el código, recompilar la aplicación y volver a distribuir el software.

Synthea adopta un enfoque completamente distinto

En Synthea el conocimiento médico se almacena en módulos independientes escritos en formato JSON.

Por ejemplo:

modules/

```
|— hypertension.json  
|— diabetes.json  
|— asthma.json  
|— pregnancy.json  
|— wellness.json
```

Estos archivos describen protocolos clínicos mediante estados y transiciones.

El motor Java **no conoce ninguna enfermedad concreta**.

Simplemente interpreta estos módulos.

Consecuencia

El motor permanece estable mientras que el conocimiento clínico puede evolucionar de forma independiente.

Esta decisión aporta una enorme flexibilidad.

Engineering Insight

Los desarrolladores de Synthea desacoplan completamente la lógica médica del motor de simulación.

En términos de ingeniería del software esto representa una clara aplicación del principio de separación de responsabilidades (*Separation of Concerns*).

2.3 Segunda decisión arquitectónica

El motor no conoce enfermedades

Esta idea suele sorprender la primera vez que se estudia el proyecto.

Muchos desarrolladores esperan encontrar clases como:

Hypertension.java

Diabetes.java

Asthma.java

Sin embargo, dichas clases no existen.

En su lugar encontramos clases genéricas como:

ConditionOnset

MedicationOrder

Encounter

Observation

Procedure

Estas clases representan acciones clínicas universales.

El tipo concreto de enfermedad viene determinado por el módulo JSON.

Por ejemplo:

```
{  
  "type": "ConditionOnset",  
  
  "code": "59621000"  
}
```

Cuando el motor encuentra este estado simplemente ejecuta:

`ConditionOnset.diagnose(...)`

No necesita saber que se trata de hipertensión.

Ventajas

Una única implementación sirve para cualquier enfermedad existente o futura.

Este diseño reduce enormemente la complejidad del sistema.

2.4 Tercera decisión arquitectónica

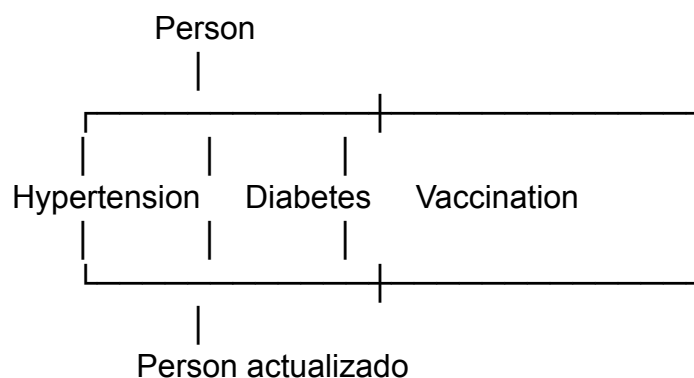
El paciente constituye el núcleo de la simulación

Toda la arquitectura gira alrededor de un único objeto.

Person

Este objeto representa el estado completo del paciente durante toda su vida.

Todos los módulos trabajan sobre la misma instancia.



Nunca existen múltiples copias del paciente.

Existe un único objeto cuya información evoluciona continuamente.

Consecuencia

Todos los módulos pueden acceder a información generada por módulos anteriores.

Por ejemplo:

Diabetes

↓

Pregunta

¿Tiene hipertensión?

La respuesta ya se encuentra almacenada en Person.

Engineering Insight

Esta decisión implementa el patrón conocido como **Single Source of Truth**.

Toda la información clínica del paciente existe en un único lugar.

2.5 Cuarta decisión arquitectónica

Separar el estado del paciente de su historia clínica

Al estudiar el código encontramos dos clases fundamentales.

Person

y

HealthRecord

Podría parecer redundante.

Sin embargo, ambas poseen responsabilidades completamente distintas.

Person

Representa el estado actual del paciente.

Por ejemplo:

- edad;
 - sexo;
 - enfermedades activas;
 - atributos;
 - situación actual.
-

HealthRecord

Representa el historial completo de eventos.

Por ejemplo:

- consultas;
 - diagnósticos;
 - procedimientos;
 - observaciones;
 - medicación.
-

Diferencia conceptual

Supongamos un paciente que tuvo una neumonía hace diez años.

Actualmente ya está curado.

Person

No tiene neumonía activa.

Sin embargo,

HealthRecord

Sí contiene el episodio histórico de neumonía.

Esta separación resulta esencial para representar correctamente una historia clínica longitudinal.

Engineering Insight

Esta decisión evita mezclar el estado actual del paciente con su historial completo.

2.6 Quinta decisión arquitectónica

Utilizar una máquina de estados

Cada módulo clínico se implementa como una máquina de estados.

Por ejemplo.

Initial

↓

Evaluate Risk

↓

Encounter

↓

ConditionOnset

↓

MedicationOrder

↓

Observation

↓

Terminal

Cada estado posee una única responsabilidad.

Cuando termina su ejecución decide cuál será el siguiente estado.

¿Por qué utilizar una máquina de estados?

Porque los protocolos clínicos son, en esencia, secuencias de decisiones.

Un protocolo puede representarse mediante preguntas.

¿Edad superior a 50 años?

↓

Sí

↓

¿Fuma?

↓

Sí

↓

Diagnóstico

Una máquina de estados reproduce exactamente este comportamiento.

Ventajas

- Fácil mantenimiento.
- Protocolos legibles.
- Posibilidad de reutilización.
- Representación natural de algoritmos clínicos.

Engineering Insight

El patrón utilizado corresponde a una implementación del patrón **State Machine**.

2.7 Sexta decisión arquitectónica

La exportación nunca modifica la simulación

Una vez finalizada la simulación comienza una segunda fase completamente distinta.

HealthRecord



FHIR Exporter



CSV Exporter



JSON Exporter

Los exportadores únicamente leen información.

Nunca modifican al paciente.

Nunca generan nuevos eventos.

Nunca ejecutan lógica clínica.

Ventaja

Puede añadirse un nuevo formato de exportación sin modificar absolutamente nada del motor de simulación.

Engineering Insight

Se aplica nuevamente el principio de responsabilidad única.

2.8 Séptima decisión arquitectónica

Arquitectura basada en eventos

Durante la simulación todo se representa mediante eventos.

Ejemplos.

ConditionOnset

↓

Nueva enfermedad
MedicationOrder

↓

Nuevo medicamento
Procedure

↓

Nuevo procedimiento

Cada evento se registra cronológicamente.

Posteriormente la historia clínica se construye simplemente ordenando dichos eventos.

2.9 Resumen de las decisiones arquitectónicas

Decisión	Motivo
Separar motor y módulos	Permitir modificar protocolos sin cambiar el motor
Módulos declarativos	Facilitar mantenimiento clínico
Person como objeto central	Mantener un único estado del paciente
HealthRecord independiente	Separar estado actual e historial
Máquina de estados	Representar protocolos clínicos
Exportadores desacoplados	Facilitar interoperabilidad
Arquitectura basada en eventos	Construir historias clínicas coherentes

2.10 ¿Qué significa todo esto para la adaptación Python?

Este capítulo proporciona una herramienta extremadamente útil para analizar cualquier adaptación de Synthea.

Cuando se reciba la implementación desarrollada por el IRYCIS no será suficiente con comprobar si existen las mismas clases.

El verdadero análisis deberá responder preguntas mucho más profundas.

- ¿Se mantiene la separación entre el motor y los módulos clínicos?
- ¿Existe todavía un objeto central equivalente a **Person**?
- ¿La historia clínica sigue almacenándose de forma independiente del estado del paciente?
- ¿Los módulos continúan representándose mediante máquinas de estados o se ha adoptado un paradigma diferente?
- ¿La exportación permanece desacoplada del proceso de simulación?

Responder a estas cuestiones permitirá evaluar si la adaptación Python conserva la filosofía arquitectónica del proyecto original o si introduce un nuevo modelo de simulación.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **Synthea separa completamente el motor de simulación del conocimiento clínico.**
 - **Las enfermedades no están implementadas como clases Java específicas; el motor ejecuta estados genéricos definidos en módulos JSON.**
 - **El objeto **Person** constituye el núcleo de toda la simulación.**
 - ****HealthRecord** almacena el historial clínico completo y se mantiene separado del estado actual del paciente.**
 - **Los módulos implementan máquinas de estados que representan protocolos clínicos.**
 - **Los exportadores transforman la información existente sin modificar la simulación.**
 - **Toda la arquitectura sigue principios clásicos de ingeniería del software: separación de responsabilidades, desacoplamiento, reutilización y diseño orientado a eventos.**
-

Engineering Notes

Principios de ingeniería identificados

- **Separation of Concerns**
- **Single Responsibility Principle (SRP)**
- **Single Source of Truth**
- **State Machine Pattern**
- **Event-Driven Architecture**
- **Loose Coupling**
- **High Cohesion**

Idea clave del capítulo

El verdadero valor de Synthea no reside en el código Java, sino en las decisiones arquitectónicas que permiten separar la simulación clínica del motor que la ejecuta.

CAPÍTULO 3

Arquitectura General del Sistema

3.1 Introducción

Toda aplicación software posee una arquitectura que define cómo colaboran sus distintos componentes.

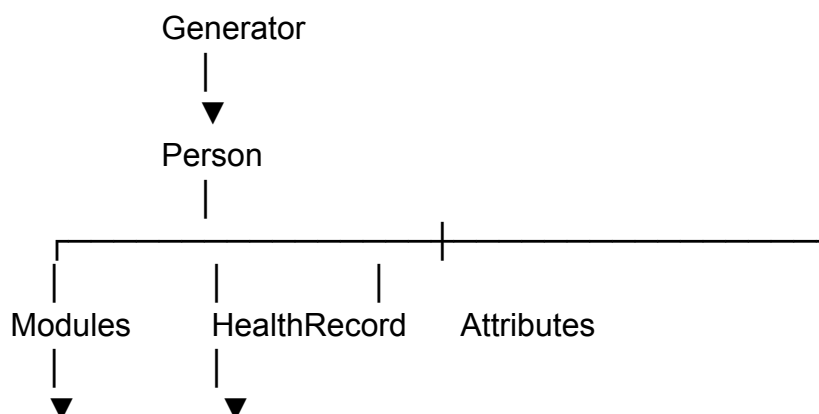
En el caso de Synthea, dicha arquitectura ha sido diseñada siguiendo un modelo claramente modular en el que cada componente posee una única responsabilidad.

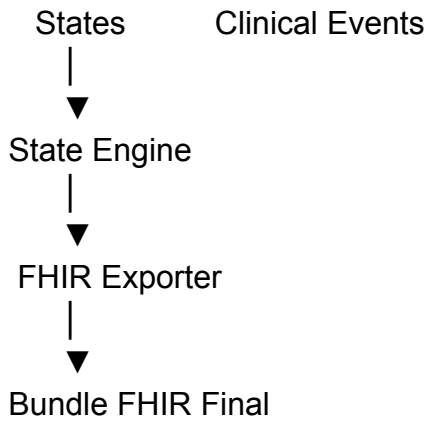
Comprender esta arquitectura resulta imprescindible antes de analizar el código fuente o cualquier adaptación posterior.

Una de las principales características de Synthea consiste en que **la simulación clínica se construye mediante la colaboración coordinada de múltiples componentes especializados**, evitando que exista una única clase responsable de todo el proceso.

3.2 Arquitectura de alto nivel

La arquitectura global de Synthea puede representarse mediante el siguiente diagrama.





Cada uno de estos bloques constituye un componente independiente del sistema.

3.3 Flujo completo de ejecución

Cuando ejecutamos Synthea mediante

`./gradlew run`

el sistema inicia un flujo perfectamente definido.

Este flujo puede resumirse de la siguiente manera.

Inicio



Generator



Creación del Person



Carga de módulos



Inicialización de estados



Simulación temporal



Actualización del paciente



Registro en HealthRecord



Exportación FHIR



Paciente generado

Aunque aparentemente solo vemos aparecer un archivo JSON en la carpeta `output`, internamente el sistema ha ejecutado cientos o miles de estados pertenecientes a distintos módulos clínicos.

3.4 El Generator

El primer componente que interviene es el **Generator**.

Su responsabilidad consiste exclusivamente en coordinar la simulación.

Generator nunca diagnostica enfermedades.

Generator nunca prescribe medicamentos.

Generator nunca modifica directamente la historia clínica.

Su única misión consiste en:

- crear la población;
- inicializar pacientes;
- cargar módulos;
- avanzar el tiempo;
- ejecutar el motor.

Por tanto:

Generator actúa como el director de la simulación, no como el responsable de las decisiones clínicas.

3.5 Creación del paciente

Generator crea un objeto Person.

Inicialmente dicho objeto contiene únicamente información básica.

Por ejemplo.

Edad = 0 años

Sexo = Hombre

Lugar = Massachusetts

Estado clínico = Saludable

Todavía no existen:

- enfermedades;
- consultas;
- medicación;
- procedimientos.

Todo ello aparecerá únicamente cuando los módulos comiencen a ejecutarse.

3.6 Carga de módulos

Una vez creado el paciente, Generator carga todos los módulos disponibles.

Por ejemplo.

modules/

hypertension.json

diabetes.json

asthma.json

covid.json

pregnancy.json

wellness.json

Cada uno representa un protocolo clínico independiente.

Es importante destacar que **todos los módulos actúan sobre el mismo paciente**.

Nunca crean copias.

Nunca crean nuevos pacientes.

3.7 Inicio de la simulación

Una vez cargados los módulos comienza la simulación.

Cada módulo inicia su ejecución desde el estado Initial.

Initial

↓

Primer estado clínico

↓

Segundo estado

↓

...

↓

Terminal

Cada estado decide cuál será el siguiente estado mediante transiciones.

3.8 Avance temporal

Una característica diferencial de Synthea consiste en que el tiempo constituye una variable fundamental.

La simulación no crea directamente un paciente de 50 años.

Por el contrario, simula su evolución.

Nacimiento



Infancia



Adolescencia



Edad adulta



Envejecimiento

En cada instante el motor evalúa si alguno de los módulos debe intervenir.

Esta aproximación permite generar historias clínicas cronológicamente coherentes.

3.9 Ejecución de estados

Cuando un módulo considera que debe actuar ejecuta uno de sus estados.

Ejemplo.

ConditionOnset

El motor localiza la implementación correspondiente dentro de State.java.

Posteriormente ejecuta:

diagnose(Person person, long time)

Este método modifica el paciente y registra el nuevo evento clínico.

3.10 Actualización del paciente

Cada estado modifica el objeto Person.

Por ejemplo.

Antes.

Person

Hipertensión = No

Después.

Person

Hipertensión = Sí

Este cambio representa el estado actual del paciente.

Sin embargo, todavía falta registrar dicho evento en la historia clínica.

3.11 Registro en HealthRecord

Una vez actualizado el paciente, el estado solicita a HealthRecord que almacene el evento.

Por ejemplo.

conditionStart(...)

HealthRecord crea una nueva entrada clínica.

Esta entrada queda asociada al Encounter correspondiente.

Es decir.

Consulta



Diagnóstico



Historia clínica

Este paso garantiza que la enfermedad queda integrada dentro de un contexto clínico coherente.

3.12 Construcción de la historia clínica

A medida que la simulación avanza, HealthRecord continúa almacenando nuevos eventos.

Nacimiento



Vacunación



Consulta



Hipertensión



Medicación

↓

Revisión

↓

Hospitalización

↓

Alta

La historia clínica aparece de forma completamente natural como consecuencia de la simulación.

3.13 Finalización de la simulación

Cuando todos los módulos han finalizado su ejecución el paciente posee una historia clínica completa.

En este momento todavía no existe ningún archivo FHIR.

Únicamente existe el objeto HealthRecord en memoria.

3.14 Exportación

El último paso consiste en convertir la historia clínica en un formato estándar.

El exportador recorre todo el contenido de HealthRecord.

Para cada evento crea un recurso FHIR equivalente.

Ejemplo.

Condition

↓

FHIR Condition

Encounter



FHIR Encounter
Medication



MedicationRequest
Procedure



FHIR Procedure

Finalmente todos los recursos se agrupan en un único Bundle.

Este Bundle constituye el archivo JSON que hemos generado durante nuestras pruebas.

3.15 Relación entre componentes

La colaboración entre los distintos componentes puede resumirse mediante el siguiente flujo.

Generator



Person



Modules



States



HealthRecord



FHIRExporter



Bundle FHIR

Cada componente recibe información del anterior y produce información para el siguiente.

En ningún momento un componente invade la responsabilidad de otro.

3.16 Aplicación de principios SOLID

La arquitectura de Synthea constituye un excelente ejemplo práctico de varios principios SOLID.

Single Responsibility Principle

Cada clase posee una única responsabilidad.

Ejemplos.

Generator coordina.

Person representa al paciente.

HealthRecord almacena historia clínica.

Exporter exporta.

Open/Closed Principle

Es posible añadir nuevos módulos clínicos sin modificar el motor.

Únicamente se incorpora un nuevo archivo JSON.

Dependency Inversion

El motor no depende de enfermedades concretas.

Depende únicamente de estados genéricos.

Separation of Concerns

La lógica clínica.

La simulación.

El almacenamiento.

La exportación.

Se encuentran completamente desacoplados.

3.17 Arquitectura observada durante el análisis

Tras el análisis realizado sobre el código fuente original pueden identificarse claramente las siguientes responsabilidades.

Componente	Responsabilidad
Generator	Coordinar la simulación
Person	Representar el paciente
Module	Describir protocolos clínicos
State	Ejecutar acciones clínicas

HealthRecord Registrar eventos
d

Exporters Generar formatos de
salida

Esta distribución de responsabilidades constituye uno de los mayores aciertos arquitectónicos del proyecto.

3.18 Implicaciones para la adaptación Python

La arquitectura descrita en este capítulo servirá como referencia durante el análisis de la adaptación desarrollada por el IRYCIS.

Más allá de comparar nombres de clases, el objetivo consistirá en responder las siguientes cuestiones.

- ¿Existe un componente equivalente a Generator?
- ¿Continúa existiendo un objeto central equivalente a Person?
- ¿Los módulos siguen siendo independientes del motor?
- ¿La historia clínica permanece separada del estado del paciente?
- ¿La exportación continúa siendo un proceso independiente?

Responder afirmativamente a estas preguntas indicaría que la adaptación conserva la filosofía arquitectónica original.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- La simulación comienza en Generator y finaliza en los exportadores FHIR.
- El paciente constituye el núcleo sobre el que actúan todos los módulos clínicos.

- **HealthRecord registra todos los eventos producidos durante la simulación.**
 - **La exportación únicamente transforma información previamente almacenada.**
 - **Cada componente posee una responsabilidad claramente definida.**
 - **La arquitectura implementa principios clásicos de ingeniería del software como SOLID, separación de responsabilidades y desacoplamiento.**
-

Engineering Notes

Decisión de diseño

Construir una arquitectura basada en componentes especializados en lugar de concentrar toda la lógica en una única clase.

Beneficios

- Mayor mantenibilidad.
- Mayor escalabilidad.
- Fácil incorporación de nuevos módulos.
- Separación clara entre simulación y exportación.
- Código altamente reutilizable.

Qué deberemos comprobar en la versión Python

- ¿Se conserva esta arquitectura por componentes?
- ¿Qué componentes desaparecen?
- ¿Qué componentes se fusionan?
- ¿Qué nuevos componentes aparecen para representar el sistema sanitario español?
- ¿El flujo Generator → Person → Modules → HealthRecord → Exporters sigue existiendo o ha sido reemplazado?

CAPÍTULO 4

El Motor de Simulación

4.1 Introducción

El componente más importante de Synthea no es el objeto `Person`, ni los módulos clínicos, ni siquiera `HealthRecord`.

El verdadero núcleo del sistema es el **motor de simulación**.

Su misión consiste en interpretar los módulos clínicos y ejecutar la evolución completa de cada paciente.

Es importante destacar que el motor **no contiene conocimiento médico**.

No sabe qué es la hipertensión.

No sabe cómo se trata una diabetes.

No conoce protocolos clínicos.

Su única responsabilidad consiste en interpretar una serie de instrucciones descritas mediante módulos JSON.

En otras palabras:

El motor es un intérprete de protocolos clínicos.

4.2 ¿Qué hace realmente el motor?

Cuando ejecutamos:

```
./gradlew run
```

Muchos desarrolladores imaginan algo parecido a esto:

Ejecutar programa



Generar paciente



Guardar FHIR

Sin embargo, el proceso real es mucho más complejo.

Leer configuración



Inicializar Generator



Crear Person



Cargar módulos



Interpretar estados



Modificar paciente



Actualizar HealthRecord



Avanzar tiempo



Repetir



Exportar resultados

El motor permanece ejecutándose durante toda la vida simulada del paciente.

4.3 Responsabilidad del motor

El motor tiene una única responsabilidad.

Coordinar la simulación completa de un paciente.

No decide enfermedades.

No prescribe medicamentos.

No calcula probabilidades.

No interpreta guías clínicas.

Todo eso pertenece a los módulos.

El motor únicamente responde a preguntas como:

- ¿Qué estado debo ejecutar ahora?
 - ¿Qué módulo corresponde?
 - ¿Ha terminado este estado?
 - ¿Cuál es el siguiente estado?
 - ¿Debo avanzar el tiempo?
-

4.4 Componentes principales del motor

Durante el análisis del código hemos identificado cuatro componentes fundamentales.

Generator



Module



State



Person

Estos cuatro objetos colaboran continuamente durante la simulación.

Generator

Coordina toda la ejecución.

Module

Representa un protocolo clínico.

State

Ejecuta una única acción clínica.

Person

Representa el estado actual del paciente.

4.5 Flujo completo de ejecución

La ejecución del motor puede resumirse mediante el siguiente algoritmo conceptual.

Crear paciente



Inicializar módulos



Entrar en Initial



Ejecutar estado



Actualizar Person



Actualizar HealthRecord



Elegir siguiente estado



¿Terminal?



No



Continuar



Sí



Finalizar módulo

Este algoritmo se repite continuamente para todos los módulos.

4.6 El papel de Generator

Generator constituye el director de orquesta del sistema.

Su función consiste en iniciar la simulación y mantenerla en funcionamiento.

Podemos imaginar su comportamiento mediante el siguiente pseudocódigo.

```
Person patient = new Person();
```

```
loadModules();
```

```
while(patient.isAlive()){
```

```
    simulate(patient);
```

```
}
```

```
export(patient);
```

Este no es el código real de Synthea.

Representa únicamente el funcionamiento conceptual del motor.

4.7 El papel del tiempo

Una de las diferencias fundamentales entre Synthea y otros sistemas consiste en que el tiempo forma parte del propio algoritmo.

No se genera un paciente de cincuenta años.

Se simulan cincuenta años.

Nacimiento

↓

1 año



2 años



...



50 años

En cada instante el motor pregunta:

¿Debe ocurrir algún evento clínico?

Si la respuesta es afirmativa, ejecuta el módulo correspondiente.

4.8 Carga de módulos

Al comenzar la simulación el motor carga todos los módulos disponibles.

Por ejemplo.

hypertension.json

diabetes.json

pregnancy.json

wellness.json

covid.json

Cada uno permanece disponible durante toda la simulación.

El motor decidirá cuándo debe intervenir cada uno.

4.9 Ejecución de estados

Cada módulo contiene una colección de estados.

Por ejemplo.

Initial



Encounter



ConditionOnset



MedicationOrder



Terminal

El motor ejecuta un estado cada vez.

Nunca ejecuta varios simultáneamente.

Cuando un estado termina devuelve el nombre del siguiente.

Por ejemplo.

"direct_transition":

"ConditionOnset"

El motor simplemente continúa.

4.10 Interpretación de estados

Cada estado posee un tipo.

Por ejemplo.

```
"type": "ConditionOnset"
```

El motor interpreta dicho tipo.

Conceptualmente realiza algo parecido a:

```
switch(type){
```

```
case ConditionOnset:
```

```
  diagnose(...)
```

```
  break;
```

```
case MedicationOrder:
```

```
  prescribe(...)
```

```
  break;
```

```
case Procedure:
```

```
  perform(...)
```

```
  break;
```

```
}
```

No importa cómo se llame el estado.

Lo importante es su tipo.

4.11 El motor nunca sabe qué enfermedad está creando

Supongamos este módulo.

```
"type": "ConditionOnset"
```

"code": "59621000"

El motor únicamente ve:

ConditionOnset

El significado del código procede del módulo.

El motor jamás pregunta.

¿Es hipertensión?

Simplemente ejecuta.

diagnose(...)

Esta decisión constituye una de las mayores fortalezas del sistema.

4.12 Comunicación entre motor y paciente

Cada estado recibe dos elementos fundamentales.

Person

time

Con esa información puede realizar cualquier modificación.

Ejemplo.

Antes.

Person

Hipertensión = No

Después.

Person

Hipertensión = Sí

Toda modificación ocurre sobre el mismo objeto.

4.13 Comunicación con HealthRecord

Cuando un estado genera un evento clínico solicita a HealthRecord que lo registre.

Por ejemplo.

```
conditionStart(...)
```

HealthRecord almacena.

Encounter

↓

Condition

↓

Procedure

↓

Observation

De esta manera el motor nunca escribe directamente sobre FHIR.

4.14 El bucle principal

Aunque el código real es mucho más complejo, conceptualmente el motor puede entenderse mediante el siguiente algoritmo.

Crear paciente

```
while(paciente vivo){
```

```
    recorrer módulos
```

ejecutar estados

actualizar paciente

registrar eventos

avanzar tiempo

}

exportar historia clínica

Todo Synthea puede entenderse como una repetición continua de este ciclo.

4.15 ¿Dónde termina el motor?

El motor termina cuando:

- el paciente fallece;
- todos los módulos han finalizado;
- se alcanza el tiempo de referencia.

En ese momento únicamente queda una tarea pendiente.

Exportar los datos.

4.16 Lo que hemos observado en el código

Durante el análisis del repositorio oficial identificamos varios aspectos importantes.

ConditionOnset no crea directamente recursos FHIR.

En su lugar llama a:

HealthRecord.conditionStart(...)

Posteriormente los exportadores convierten dicho evento en un recurso FHIR.

Este flujo confirma la separación existente entre:

- simulación;
 - almacenamiento;
 - exportación.
-

4.17 El verdadero papel del motor

Una idea muy frecuente consiste en pensar que el motor genera enfermedades.

No es correcto.

El motor genera **transiciones**.

Las enfermedades constituyen únicamente una consecuencia de dichas transiciones.

Podemos resumirlo mediante el siguiente diagrama.

Motor



Estado



Transición



Cambio en Person



Nuevo evento clínico



HealthRecord



4.18 Implicaciones para la adaptación Python

Este capítulo proporciona probablemente la herramienta más importante para analizar la adaptación desarrollada por el IRYCIS.

Cuando recibamos el código Python no deberemos buscar únicamente equivalencias entre clases.

El verdadero análisis consistirá en responder las siguientes preguntas.

- ¿Existe un bucle principal equivalente al del motor Java?
- ¿Cómo interpreta Python los módulos clínicos?
- ¿Los estados siguen ejecutándose mediante una máquina de estados?
- ¿Cómo se representa el paso del tiempo?
- ¿Existe un equivalente funcional de **Generator**?
- ¿Cómo se comunica el motor con el objeto que representa al paciente?

Responder a estas cuestiones permitirá determinar si la adaptación constituye una simple traducción del motor o una reimplementación con un paradigma diferente.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- El motor no contiene conocimiento clínico.
- El motor interpreta protocolos clínicos descritos mediante módulos JSON.
- Cada estado ejecuta una única acción sobre el paciente.
- Todas las modificaciones se realizan sobre un único objeto **Person**.
- HealthRecord registra los eventos generados durante la simulación.
- FHIR únicamente representa el resultado final del proceso.
- La simulación puede entenderse como un ciclo continuo de interpretación de estados y actualización del paciente.

Engineering Notes

Patrón arquitectónico identificado

Interpreter Pattern

El motor actúa como un intérprete que ejecuta instrucciones descritas mediante un lenguaje declarativo (los módulos JSON).

Patrones de diseño presentes

- Interpreter Pattern
 - State Machine
 - Single Responsibility Principle
 - Strategy (mediante módulos independientes)
 - Loose Coupling
-

Idea fundamental del capítulo

El motor de Synthea no sabe medicina; sabe interpretar protocolos clínicos. Toda la inteligencia médica reside en los módulos.

CAPÍTULO 5

Person: El núcleo de la simulación

5.1 Introducción

Durante el análisis del código fuente puede parecer que la clase más importante del proyecto debería ser **Generator**, ya que inicia la simulación, o **State**, ya que ejecuta las acciones clínicas.

Sin embargo, ambas clases tienen una función instrumental.

El verdadero protagonista del sistema es el objeto **Person**.

Toda la arquitectura de Synthea gira alrededor de esta clase.

Todos los módulos la modifican.

Todos los estados reciben una instancia de Person.

Toda la historia clínica pertenece a Person.

Cuando el paciente fallece, la simulación termina porque deja de existir una evolución posible para dicho objeto.

Por tanto,

Person constituye el eje central de toda la simulación clínica.

5.2 ¿Qué representa realmente Person?

Una interpretación superficial sería afirmar:

"Person representa un paciente."

Sin embargo, esta afirmación es incompleta.

La definición correcta sería la siguiente.

Person representa el estado completo de un individuo durante toda su vida simulada.

Esto implica que Person contiene tanto la situación clínica actual del paciente como toda la información necesaria para que el motor pueda continuar simulando su evolución.

No representa únicamente datos.

Representa un proceso dinámico.

5.3 Person como objeto de estado

Desde el punto de vista de la ingeniería del software, Person puede entenderse como un **State Object**.

Es decir, un objeto cuyo contenido evoluciona continuamente conforme avanza la simulación.

Podemos imaginar su evolución del siguiente modo.

Nacimiento

↓

Edad = 0

↓

Edad = 15

↓

Fumador = Sí

↓

Hipertensión = Sí

↓

Tratamiento = Enalapril

↓

Infarto

↓

Seguimiento

↓

Fallecimiento

El objeto sigue siendo el mismo.

Únicamente cambia su estado interno.

5.4 ¿Qué información almacena?

Aunque el contenido exacto puede evolucionar entre versiones, conceptualmente Person almacena cinco grandes grupos de información.

Información demográfica

Corresponde a las características básicas del individuo.

Incluye elementos como:

- nombre;
- sexo;
- fecha de nacimiento;
- dirección;
- raza;
- etnia;
- estado civil.

Esta información suele permanecer estable durante gran parte de la simulación.

Estado clínico

Representa la situación médica actual.

Ejemplos.

- enfermedades activas;

- embarazo;
- tabaquismo;
- obesidad;
- hipertensión;
- diabetes.

Este estado cambia continuamente.

Atributos internos

Uno de los aspectos más interesantes de Person es la existencia de un conjunto de atributos utilizados por el propio motor.

Por ejemplo.

Cuando analizamos ConditionOnset observamos:

```
person.attributes.put(assignToAttribute, entry);
```

Esto significa que Person actúa también como un repositorio interno de información utilizada por otros módulos.

Ejemplo.

hypertension_dx

↓

Referencia al diagnóstico

Posteriormente otro módulo puede consultar dicho atributo.

Esta característica permite la comunicación entre módulos completamente independientes.

Historia clínica

Person mantiene una referencia a HealthRecord.

No almacena directamente los recursos FHIR.

Mantiene una referencia al objeto encargado de registrar todos los eventos clínicos.

Información administrativa

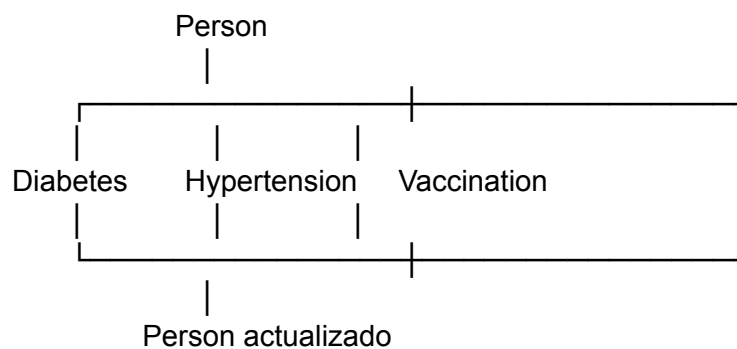
También almacena información relacionada con:

- seguros;
- proveedores sanitarios;
- médicos;
- costes;
- localización.

Esta información resulta especialmente importante para la simulación del sistema sanitario estadounidense.

5.5 Person como punto de encuentro

Todos los módulos trabajan sobre el mismo objeto.



Esta decisión arquitectónica evita inconsistencias.

Todos los módulos observan exactamente el mismo estado del paciente.

5.6 Comunicación entre módulos

Supongamos dos módulos.

Hypertension

↓

Diagnóstico

y

Cardiovascular Disease

El segundo módulo puede preguntar.

¿Tiene hipertensión?

La respuesta ya se encuentra almacenada dentro de Person.

No necesita volver a calcularla.

Esto convierte a Person en el mecanismo principal de comunicación entre módulos.

5.7 Person no toma decisiones

Este punto resulta extremadamente importante.

Person nunca decide nada.

No diagnostica.

No prescribe.

No calcula probabilidades.

No interpreta protocolos.

Únicamente almacena información.

Las decisiones pertenecen exclusivamente a los estados ejecutados por el motor.

Podemos resumirlo así.

State

↓

Decisión

↓

Person

↓

Nuevo estado

5.8 Relación con HealthRecord

Existe una diferencia fundamental entre ambos componentes.

Person representa el presente.

HealthRecord representa el pasado.

Ejemplo.

Paciente.

Hipertensión = No

Pero.

HealthRecord

↓

Hipertensión en 2014

La enfermedad ya no está activa.

Sin embargo continúa formando parte de la historia clínica.

Esta separación constituye una de las decisiones arquitectónicas más elegantes del proyecto.

5.9 Ciclo de vida de Person

El objeto Person existe durante toda la simulación.

Su ciclo puede resumirse del siguiente modo.

Generator

↓

Crear Person

↓

Inicializar atributos

↓

Ejecutar módulos

↓

Actualizar estado

↓

Registrar eventos

↓

Exportar historia clínica

↓

Finalizar simulación

Una única instancia acompaña al paciente durante toda su vida.

5.10 Person y la simulación

La mejor forma de entender Person consiste en imaginar un videojuego.

Existe un único personaje.

Cada nivel modifica su estado.

Nunca aparece un personaje nuevo.

Synthea funciona exactamente igual.

Cada módulo modifica el mismo paciente.

Por tanto.

La simulación consiste simplemente en observar cómo evoluciona el estado interno del objeto Person conforme el motor ejecuta nuevos estados.

5.11 Observaciones obtenidas durante el análisis del código

Durante el análisis del código Java identificamos varios aspectos relevantes.

Person aparece como argumento en prácticamente todos los estados.

Ejemplo.

`diagnose(Person person, long time)`

Esto confirma que el paciente constituye el elemento compartido por todo el sistema.

Además observamos el uso intensivo de:

`person.attributes`

como mecanismo de comunicación entre módulos.

Esta característica será especialmente interesante cuando analicemos la implementación Python.

5.12 ¿Qué deberíamos buscar en la versión Python?

Cuando recibamos la adaptación desarrollada por el IRYCIS deberemos localizar inmediatamente el equivalente funcional de Person.

No importa cómo se llame.

Puede denominarse.

Patient

ClinicalPatient

SyntheticPatient

Citizen

Person

Lo importante no será el nombre.

Lo importante será comprobar:

- ¿Existe un único objeto central?
- ¿Todos los módulos modifican esa misma instancia?
- ¿Se mantiene la separación respecto a la historia clínica?
- ¿Continúan existiendo atributos internos compartidos?

Responder a estas preguntas permitirá evaluar si la filosofía arquitectónica original se mantiene.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **Person** constituye el núcleo de toda la simulación.
 - Representa el estado completo del paciente y no únicamente sus datos demográficos.
 - Todos los módulos trabajan sobre una única instancia compartida.
 - **Person** actúa como mecanismo de comunicación entre módulos.
 - Las decisiones clínicas no pertenecen a **Person**, sino a los estados del motor.
 - **HealthRecord** mantiene el historial clínico mientras **Person** representa el estado actual del paciente.
-

Engineering Notes

Patrón de diseño identificado

State Object

Person representa el estado completo del sistema en un instante determinado.

Responsabilidades

Responsabilidad	¿Pertenece a Person?
Representar al paciente	Sí
Mantener atributos internos	Sí

Compartir información entre módulos	Sí
Registrar historia clínica	No (HealthRecord)
Diagnosticar enfermedades	No (State)
Prescribir medicación	No (State)
Exportar FHIR	No (Exporter)

Idea fundamental del capítulo

Person no representa simplemente un paciente. Representa el estado completo de una simulación clínica. Todo Synthea gira alrededor de la evolución continua de este único objeto.

IMPORTANTE

La mayoría de personas abrirán la versión Python buscando clases parecidas a `Person.java`.

¿Cuál es el objeto que concentra el estado completo de la simulación?

Puede llamarse `Patient`, `Citizen`, `ClinicalRecord` o cualquier otra cosa.

El nombre es irrelevante.

Lo importante es identificar **el papel arquitectónico** que desempeña.

CAPÍTULO 6

Los Módulos Clínicos: El verdadero cerebro de Synthea

6.1 Introducción

Una de las decisiones de diseño más importantes del proyecto consiste en separar completamente el conocimiento clínico del motor de simulación.

Mientras que el motor implementa únicamente la lógica necesaria para interpretar estados y coordinar la simulación, **todo el conocimiento médico reside en los módulos clínicos.**

Esta separación convierte a Synthea en una plataforma altamente extensible.

Añadir una nueva enfermedad no requiere modificar el motor.

Basta con crear un nuevo módulo.

6.2 ¿Qué es un módulo?

La definición más habitual es:

"Un módulo representa una enfermedad."

Esta definición es incorrecta.

La definición correcta es:

Un módulo representa un proceso clínico completo.

Ese proceso puede corresponder a:

- una enfermedad;

- un protocolo asistencial;
- una vacunación;
- un embarazo;
- una atención en urgencias;
- un seguimiento médico;
- un proceso de cuidados paliativos.

Por tanto, un módulo describe **cómo evoluciona un proceso sanitario**, no simplemente una enfermedad.

6.3 Organización de los módulos

Todos los módulos se encuentran dentro del directorio.

src/main/resources/modules/

Por ejemplo.

modules/

hypertension.json

diabetes.json

asthma.json

wellness.json

pregnancy.json

covid19.json

Cada archivo constituye un protocolo completamente independiente.

El motor únicamente necesita conocer su existencia.

No necesita conocer su contenido.

6.4 Anatomía de un módulo

Todos los módulos siguen prácticamente la misma estructura.

```
{  
  
"name":"Hypertension",  
  
"remarks":[...]  
  
"states":{  
  
...  
  
}  
  
}
```

Podemos dividir un módulo en tres grandes bloques.

Información descriptiva

"name"

Identifica el módulo.

Ejemplo.

Hypertension

Documentación

"remarks"

Este apartado contiene referencias bibliográficas, explicaciones clínicas y justificación del protocolo implementado.

Durante el análisis del módulo de hipertensión observamos referencias a:

- ESC/ESH Guidelines
- ACC/AHA Guidelines
- Canadian Hypertension Guidelines

Esto demuestra que el módulo ha sido construido siguiendo evidencia científica.

Estados

"states"

Este bloque contiene la lógica ejecutable.

Todo lo anterior constituye únicamente documentación.

Todo lo que aparece dentro de "states" será interpretado por el motor.

6.5 Los estados

Cada módulo está formado por una colección de estados.

Por ejemplo.

Initial

↓

Assign Probability

↓

ConditionOnset

↓

MedicationOrder

↓

Observation

↓

Terminal

Cada estado representa una única acción.

Nunca realiza varias acciones simultáneamente.

Este principio constituye una aplicación directa del **Single Responsibility Principle**.

6.6 El estado Initial

Todos los módulos comienzan aquí.

Ejemplo real.

```
"Initial":{  
  
  "type":"Initial",  
  
  "direct_transition":  
  
    "Assign Hypertension Base Probability"  
  
}
```

Este estado no realiza ninguna acción clínica.

Simplemente indica al motor dónde comienza el protocolo.

Podemos interpretarlo como:

Entrar en el módulo

↓

Comenzar protocolo

6.7 Las transiciones

Una vez finaliza un estado, el motor debe decidir cuál será el siguiente.

Existen varios tipos de transición.

Direct Transition

La transición más sencilla.

"direct_transition":

"ConditionOnset"

El motor continúa inmediatamente hacia ese estado.

Conditional Transition

Permite tomar decisiones.

Ejemplo conceptual.

Edad > 50

↓

Sí

↓

ConditionOnset

↓

No

↓

Terminal

Distributed Transition

Permite introducir probabilidades.

Ejemplo.

80%

↓

Estado A

20%



Estado B

Este tipo de transición constituye la base de la simulación probabilística.

Complex Transition

Permite combinar múltiples condiciones.

Es utilizada en protocolos clínicos especialmente complejos.

6.8 Los tipos de estado

El nombre del estado puede ser cualquiera.

Por ejemplo.

Diagnose_Hypertension

o

State12

El motor ignora ese nombre.

Lo importante es el tipo.

"type": "ConditionOnset"

Es el tipo quien determina el comportamiento.

Estados más importantes

Initial

Punto de entrada del módulo.

Encounter

Genera una consulta médica.

Observation

Registra una observación clínica.

ConditionOnset

Genera un diagnóstico.

ConditionEnd

Finaliza una enfermedad.

MedicationOrder

Prescribe medicación.

Procedure

Realiza un procedimiento.

Delay

Hace avanzar el tiempo.

Terminal

Finaliza el módulo.

6.9 Ejemplo real

Durante nuestro análisis estudiamos el siguiente fragmento.

```
"Diagnose_Hypertension":{  
  
  "type":"ConditionOnset",  
  
  "codes":[...]  
  
}
```

Cuando el motor encuentra este estado ocurre exactamente lo siguiente.

JSON

↓

ConditionOnset

↓

diagnose()

↓

HealthRecord.conditionStart()

↓

Condition FHIR

Este flujo conecta directamente el módulo con la historia clínica.

6.10 Los módulos no ejecutan código

Este aspecto resulta fundamental.

Los módulos **no contienen Java**.

No contienen funciones.

No contienen algoritmos imperativos.

Únicamente describen qué debe ocurrir.

El motor se encarga de ejecutar dichas instrucciones.

En otras palabras.

JSON

↓

Describe

↓

Motor

↓

Ejecuta

Esta separación constituye una de las mayores fortalezas de Synthea.

6.11 Los módulos pueden colaborar

Supongamos el siguiente escenario.

El módulo de diabetes pregunta.

¿Tiene hipertensión?

No necesita conocer el módulo de hipertensión.

Simplemente consulta Person.

Si Person contiene dicha información, el módulo puede adaptar su comportamiento.

Este mecanismo permite que módulos completamente independientes colaboren entre sí.

6.12 ¿Qué ocurriría si quisiéramos adaptar España?

Este punto conecta directamente con el proyecto del IRYCIS.

Supongamos que queremos introducir Atención Primaria antes del diagnóstico de hipertensión.

Actualmente el protocolo simplificado podría ser.

Initial



ConditionOnset



MedicationOrder

La adaptación española podría transformarlo en.

Initial



Primary Care Encounter



Evaluación



ConditionOnset



Seguimiento



MedicationOrder

Obsérvese que el motor no cambia.

Únicamente cambia el protocolo.

Esta idea será fundamental cuando comparemos la implementación Python.

6.13 Ventajas de la arquitectura basada en módulos

Esta decisión aporta múltiples ventajas.

Reutilización

El motor puede ejecutar cualquier módulo.

Extensibilidad

Añadir una nueva enfermedad no requiere modificar Java.

Mantenimiento

Los protocolos clínicos evolucionan independientemente del motor.

Legibilidad

Los módulos pueden ser comprendidos por clínicos con conocimientos mínimos de programación.

Validación

Resulta sencillo comparar un protocolo con la guía clínica correspondiente.

6.14 Limitaciones

Este diseño también presenta algunos inconvenientes.

Protocolos extremadamente complejos pueden resultar difíciles de representar exclusivamente mediante estados.

Además, cualquier nuevo tipo de estado requiere modificar el motor.

Por ejemplo.

Si deseáramos crear.

ReferralToPrimaryCare

El motor debería aprender a interpretar dicho estado.

En ese caso ya no bastaría con modificar el JSON.

6.15 Implicaciones para la adaptación Python

Este capítulo probablemente sea el más importante para nuestro proyecto.

Cuando recibamos el código del IRYCIS deberemos responder inmediatamente las siguientes preguntas.

- ¿Siguen existiendo módulos?
- ¿Continúan representándose mediante JSON?
- ¿Se han convertido en clases Python?
- ¿La lógica clínica sigue separada del motor?
- ¿Las adaptaciones españolas se implementan modificando módulos o modificando el motor?

Responder a estas cuestiones permitirá comprender la filosofía de la adaptación.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- Los módulos constituyen el verdadero conocimiento clínico de Synthea.
- Un módulo representa un proceso clínico completo y no simplemente una enfermedad.
- Cada módulo está formado por una máquina de estados.
- Los estados describen acciones clínicas independientes.
- El motor interpreta los módulos sin conocer su significado médico.
- La separación entre motor y módulos constituye la decisión arquitectónica más importante del proyecto.
- Las adaptaciones nacionales pueden implementarse modificando los módulos sin alterar necesariamente el motor.

Engineering Notes

Patrón de diseño identificado

Data-Driven Architecture

El comportamiento del sistema viene determinado por datos (JSON) y no por código específico.

Patrones presentes

- Interpreter Pattern
 - State Machine
 - Data-Driven Design
 - Separation of Concerns
 - Loose Coupling
-

Idea fundamental del capítulo

Los módulos no contienen código ejecutable. Contienen conocimiento clínico. El motor simplemente interpreta ese conocimiento y lo convierte en una simulación.

CAPÍTULO 7

La Máquina de Estados: Cómo piensa Synthea

7.1 Introducción

Tras estudiar la arquitectura general del sistema y el funcionamiento de los módulos clínicos, aparece una pregunta inevitable.

¿Cómo ejecuta realmente un módulo el motor de simulación?

La respuesta reside en uno de los patrones de diseño más utilizados en ingeniería del software.

State Machine (Máquina de Estados).

Toda la simulación de Synthea se basa en este patrón.

Cada protocolo clínico puede entenderse como una secuencia de estados conectados mediante transiciones.

El motor únicamente recorre dichas transiciones.

7.2 ¿Qué es una máquina de estados?

Una máquina de estados es un modelo matemático utilizado para representar procesos cuyo comportamiento depende del estado actual del sistema.

En ingeniería del software se emplea para modelar procesos secuenciales donde cada paso determina cuál será el siguiente.

En Synthea, cada módulo clínico constituye una máquina de estados independiente.

Por ejemplo.

Initial



Encounter



ConditionOnset



MedicationOrder



Observation



Terminal

Cada rectángulo representa un estado.

Cada flecha representa una transición.

7.3 ¿Por qué utilizar una máquina de estados?

Los protocolos clínicos poseen una estructura muy similar a una máquina de estados.

Por ejemplo.

Un protocolo de hipertensión podría describirse verbalmente del siguiente modo.

El paciente acude a consulta.



Se mide la presión arterial.

↓

Si cumple criterios diagnósticos...

↓

Se diagnostica hipertensión.

↓

Se prescribe tratamiento.

↓

Se programa seguimiento.

Obsérvese que cada paso depende del anterior.

Esta estructura encaja perfectamente con una máquina de estados.

7.4 Estado frente a evento

Este punto suele generar mucha confusión.

Es importante distinguir claramente ambos conceptos.

Estado

Representa una acción que el motor debe ejecutar.

Ejemplos.

- Initial
 - Encounter
 - ConditionOnset
 - Observation
 - MedicationOrder
-

Evento

Representa el resultado producido tras ejecutar un estado.

Ejemplo.

Estado.

ConditionOnset

Evento generado.

Diagnóstico de hipertensión.

Otro ejemplo.

Estado.

MedicationOrder

Evento generado.

Prescripción de Enalapril.

Por tanto.

Los estados producen eventos.

7.5 Anatomía de un estado

Durante el análisis de `hypertension.json` encontramos un ejemplo representativo.

```
"Diagnose_Hypertension":{  
  
  "type":"ConditionOnset",  
  
  "codes":[...]  
  
}
```

Este estado contiene cuatro elementos fundamentales.

Nombre

Diagnose_Hypertension

Sirve únicamente para identificar el estado.

El motor no utiliza este nombre para decidir cómo ejecutarlo.

Tipo

ConditionOnset

Es el elemento realmente importante.

Determina qué clase Java será utilizada.

Parámetros

En este caso.

SNOMED

59621000

El estado proporciona la información necesaria para crear el diagnóstico.

Transición

Una vez ejecutado.

El motor continuará hacia otro estado.

7.6 Ciclo de vida de un estado

Todos los estados siguen exactamente el mismo ciclo.

Motor

↓

Leer estado

↓

Interpretar tipo

↓

Ejecutar acción

↓

Modificar Person

↓

Actualizar HealthRecord

↓

Elegir siguiente estado

Este ciclo se repite miles de veces durante una simulación.

7.7 El motor nunca ejecuta módulos completos

Este aspecto resulta extremadamente importante.

El motor nunca hace algo como:

Ejecutar módulo hipertensión

Lo que realmente hace es.

Ejecutar Initial

↓

Ejecutar Encounter



Ejecutar Observation



Ejecutar ConditionOnset



Ejecutar MedicationOrder

Es decir.

El motor únicamente conoce estados.

Nunca protocolos completos.

7.8 Tipos principales de estados

Durante el estudio del código fuente identificamos los siguientes tipos principales.

Estado	Responsabilidad
Initial	Comenzar módulo
Encounter	Crear consulta
Observation	Registrar observación
ConditionOnset	Diagnosticar enfermedad
ConditionEnd	Finalizar enfermedad
MedicationOrder	Prescribir medicamento
Procedure	Registrar procedimiento

Delay	Avanzar el tiempo
Terminal	Finalizar módulo

Todos ellos se encuentran implementados dentro de `State.java`.

7.9 Las transiciones

Una vez ejecutado un estado el motor debe decidir cuál será el siguiente.

Existen cuatro mecanismos principales.

Direct Transition

"direct_transition":

"MedicationOrder"

El flujo continúa sin condiciones.

Conditional Transition

El siguiente estado depende de una condición.

¿Edad superior a 65?

↓

Sí

↓

MedicationOrder

↓

No

↓

Terminal

Distributed Transition

El siguiente estado depende de una probabilidad.

80%

↓

Estado A

20%

↓

Estado B

Complex Transition

Permite combinar múltiples reglas clínicas.

Es utilizada en módulos especialmente complejos.

7.10 El tiempo también es un estado

Uno de los aspectos más elegantes del diseño de Synthea consiste en que el tiempo no pertenece al motor.

Pertenece a los estados.

Ejemplo.

Delay

Cuando el motor ejecuta Delay.

Simplemente avanza el tiempo.

Después continúa con el siguiente estado.

Esto permite representar fácilmente.

- seguimientos.
 - revisiones.
 - controles periódicos.
 - tratamientos prolongados.
-

7.11 Seguimiento completo de un estado

Tomemos como ejemplo ConditionOnset.

El recorrido completo es el siguiente.

JSON

↓

ConditionOnset

↓

State.java

↓

diagnose()

↓

Person

↓

HealthRecord

↓

FHIR Condition

Todo comienza con un único estado.

7.12 ¿Qué ocurre cuando finaliza un estado?

Una vez terminada su ejecución.

El estado nunca vuelve atrás.

El motor simplemente pregunta.

¿Cuál es el siguiente estado?

Y continúa.

Esto convierte la simulación en un recorrido continuo por un grafo dirigido.

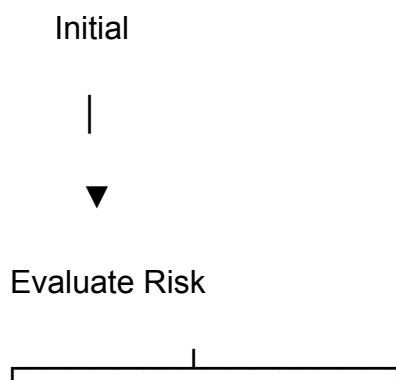
7.13 Los módulos son grafos

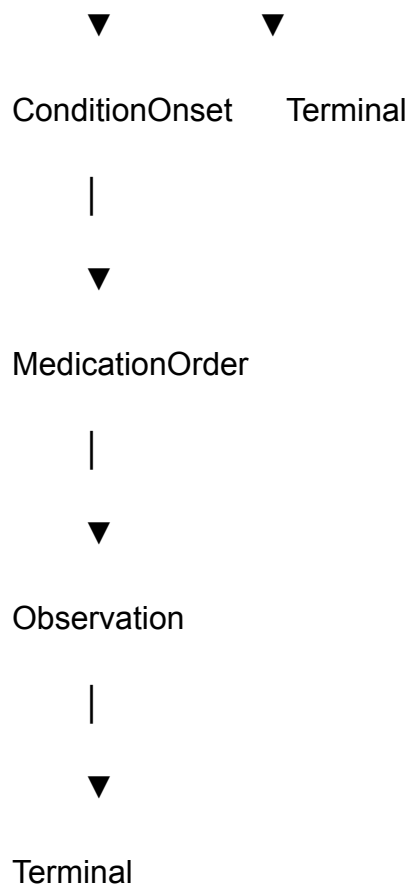
Este es probablemente uno de los conceptos más importantes del libro.

Desde un punto de vista matemático.

Un módulo no es un algoritmo.

Es un grafo.





El motor únicamente recorre dicho grafo.

7.14 Relación con el patrón State

El patrón State clásico describe sistemas cuyo comportamiento depende del estado actual.

Synthea lleva este patrón un paso más allá.

Cada estado constituye además un objeto ejecutable.

Esto permite encapsular completamente cada acción clínica.

7.15 Ventajas del diseño

La utilización de máquinas de estados aporta múltiples ventajas.

Claridad

Los protocolos pueden representarse gráficamente.

Reutilización

El mismo motor interpreta cualquier protocolo.

Extensibilidad

Nuevos módulos pueden añadirse sin modificar el motor.

Validación

Los clínicos pueden revisar protocolos sin necesidad de comprender Java.

Depuración

Cada estado constituye una unidad independiente.

7.16 Limitaciones

No todos los procesos clínicos son fácilmente representables mediante estados.

Protocolos extremadamente complejos pueden requerir nuevos tipos de estado.

En ese caso el motor debe modificarse.

7.17 Implicaciones para la adaptación Python

Este capítulo será especialmente útil cuando analicemos el proyecto del IRYCIS.

Las preguntas fundamentales serán.

- ¿Continúa existiendo una máquina de estados?
- ¿Los estados siguen siendo objetos independientes?
- ¿Los módulos continúan representándose como grafos?
- ¿El flujo sigue siendo dirigido por transiciones?
- ¿Python interpreta los JSON o implementa directamente la lógica mediante clases?

Responder a estas preguntas permitirá determinar si la filosofía original del motor se mantiene.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- Cada módulo constituye una máquina de estados.
 - Los estados representan acciones clínicas independientes.
 - Los eventos clínicos son el resultado de ejecutar dichos estados.
 - El motor nunca ejecuta un protocolo completo; únicamente ejecuta estados.
 - Las transiciones determinan el recorrido seguido por la simulación.
 - Desde un punto de vista matemático, un módulo puede entenderse como un grafo dirigido.
-

Engineering Notes

Patrones de diseño identificados

- Finite State Machine

- **State Pattern**
 - **Graph Traversal**
 - **Interpreter Pattern**
-

Idea fundamental del capítulo

Synthea no ejecuta protocolos médicos. Recorre grafos de estados que representan dichos protocolos.

CAPÍTULO 8

HealthRecord: La Historia Clínica Longitudinal

8.1 Introducción

Una vez comprendido el funcionamiento del motor de simulación, los módulos clínicos y el objeto Person, surge una pregunta fundamental.

¿Dónde se almacena toda la información generada durante la simulación?

La respuesta es **HealthRecord**.

HealthRecord constituye uno de los componentes más importantes de toda la arquitectura de Synthea.

Su función consiste en registrar cronológicamente todos los eventos clínicos producidos durante la vida del paciente.

Es importante destacar que **HealthRecord no genera información**.

Únicamente la almacena.

8.2 ¿Qué es realmente HealthRecord?

Una definición habitual sería:

"Es la historia clínica del paciente."

Aunque correcta, resulta incompleta.

La definición más precisa sería:

HealthRecord representa el repositorio estructurado donde se almacenan todos los eventos clínicos generados por el motor de simulación durante la vida de un paciente.

No representa únicamente enfermedades.

Representa absolutamente toda la actividad clínica.

8.3 Responsabilidad de HealthRecord

HealthRecord posee una única responsabilidad.

Registrar cronológicamente los eventos clínicos generados durante la simulación.

No diagnostica.

No prescribe.

No decide.

No calcula probabilidades.

Únicamente registra.

Esto constituye un excelente ejemplo del **Single Responsibility Principle**.

8.4 ¿Qué almacena?

Durante el análisis del código fuente observamos que HealthRecord almacena múltiples tipos de información.

Entre ellos.

Encuentros clínicos

Encounter

Representan consultas.

Hospitalizaciones.

Urgencias.

Atención Primaria.

Condiciones

Condition

Representan diagnósticos.

Por ejemplo.

- hipertensión;
 - diabetes;
 - obesidad.
-

Procedimientos

Procedure

Representan intervenciones médicas.

Medicación

Medication

Representa tratamientos farmacológicos.

Observaciones

Observation

Representan constantes vitales.

Resultados analíticos.

Mediciones.

Care Plans

Representan planes terapéuticos.

Claims

Representan información económica.

Especialmente importante en el sistema sanitario estadounidense.

8.5 HealthRecord no representa el estado actual

Este aspecto resulta absolutamente fundamental.

Supongamos el siguiente escenario.

Paciente.

2012

↓

Hipertensión

↓

Tratamiento

↓

Control

↓

Curación

Actualmente.

Person

Hipertensión = No

Sin embargo.

HealthRecord



Diagnóstico de hipertensión



Tratamiento



Seguimiento



Fin del episodio

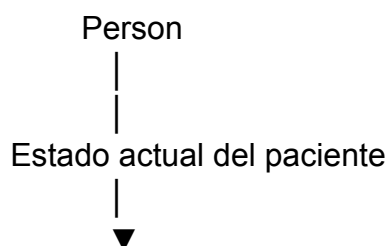
La enfermedad desaparece del estado actual del paciente, pero nunca desaparece de la historia clínica.

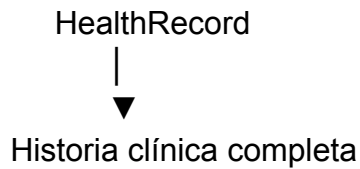
Esta diferencia constituye una de las decisiones arquitectónicas más importantes de todo Synthea.

8.6 Relación entre Person y HealthRecord

Ambos componentes colaboran continuamente.

Podemos representarlo mediante el siguiente diagrama.





Person representa el presente.

HealthRecord representa toda la evolución histórica.

8.7 Análisis del método `conditionStart()`

Durante nuestro estudio del código analizamos el siguiente método.

`conditionStart(...)`

Este método constituye el punto exacto donde nace una enfermedad dentro de la historia clínica.

Su funcionamiento puede resumirse del siguiente modo.

ConditionOnset



`conditionStart()`



Crear Entry



Buscar Encounter activo



Añadir Condition



Actualizar lista de condiciones activas

Es importante observar que **ConditionOnset nunca crea directamente un recurso FHIR**.

Únicamente solicita a HealthRecord que registre un nuevo evento clínico.

8.8 El concepto de Entry

Durante el análisis encontramos la siguiente instrucción.

```
Entry condition = new Entry(...)
```

Entry constituye la representación interna de cualquier elemento clínico almacenado en HealthRecord.

Podemos entenderlo como la unidad mínima de información clínica.

Dependiendo del contexto puede representar.

- una enfermedad;
- un procedimiento;
- una observación;
- una medicación.

Posteriormente los exportadores decidirán cómo transformar dicha Entry en recursos FHIR.

8.9 La lista de condiciones activas

Uno de los mecanismos más elegantes observados durante el análisis consiste en el uso de.

present

Esta estructura almacena únicamente las enfermedades activas.

Conceptualmente puede representarse así.

```
present = {
```

"59621000": hipertension,

"44054006": diabetes

}

Cuando aparece una nueva enfermedad.

`present.put(...)`

Cuando desaparece.

`present.remove(...)`

Obsérvese que eliminarla de `present` **no implica eliminarla de la historia clínica**.

Simplemente deja de estar activa.

8.10 Relación con Encounter

Otro aspecto especialmente elegante consiste en que las enfermedades nunca aparecen aisladas.

Durante el análisis observamos.

`Encounter encounter = currentEncounter(...)`

Posteriormente.

`encounter.conditions.add(condition)`

Esto significa que toda enfermedad pertenece siempre a una consulta concreta.

Desde el punto de vista clínico esta decisión resulta completamente coherente.

Un diagnóstico siempre ocurre durante una interacción asistencial.

8.11 Relación con Claim

Durante el análisis también apareció la siguiente línea.

encounter.claim.addLineItem(condition)

Esta instrucción añade automáticamente el diagnóstico a la factura clínica.

Esta decisión posee sentido dentro del sistema sanitario estadounidense.

Sin embargo.

Constituye uno de los primeros candidatos a ser modificado durante una adaptación al sistema sanitario español.

8.12 Construcción progresiva de la historia clínica

HealthRecord no se genera de una sola vez.

Se construye continuamente.

Nacimiento

↓

Vacunación

↓

Consulta

↓

Diagnóstico

↓

Tratamiento

↓

Seguimiento

↓

Hospitalización



Alta

Cada nuevo evento amplía la historia clínica existente.

8.13 Relación con FHIR

Una vez finalizada la simulación.

FHIRExporter recorre HealthRecord.

Cada Entry se convierte en uno o varios recursos.

Por ejemplo.

Entry



Condition



FHIR Condition
Encounter



FHIR Encounter
Medication



MedicationRequest

Es importante comprender que FHIR constituye únicamente una representación.

La verdadera historia clínica reside en HealthRecord.

8.14 Decisiones arquitectónicas identificadas

Durante el análisis del código podemos identificar varias decisiones especialmente interesantes.

Separación entre estado e historial

Person representa el presente.

HealthRecord representa el pasado.

Organización cronológica

Todos los eventos quedan ordenados temporalmente.

Asociación mediante Encounter

Toda actividad clínica pertenece a una consulta.

Desacoplamiento respecto a FHIR

HealthRecord no depende del estándar FHIR.

Esto permite añadir nuevos exportadores sin modificar la historia clínica.

8.15 Implicaciones para la adaptación Python

Este capítulo será especialmente útil cuando analicemos la adaptación del IRYCIS.

Las preguntas fundamentales serán.

- **¿Existe un equivalente funcional de HealthRecord?**
- **¿Cómo se almacenan los eventos clínicos?**
- **¿Se mantiene la separación entre Person e historial?**
- **¿Los diagnósticos siguen vinculados a Encounter?**
- **¿Cómo se representan los costes sanitarios en el contexto español?**
- **¿Qué sustituye al modelo de Claims estadounidense?**

Responder a estas cuestiones permitirá evaluar la fidelidad de la adaptación respecto al diseño original.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **HealthRecord constituye el repositorio central de todos los eventos clínicos.**
 - **No representa el estado actual del paciente, sino su historia completa.**
 - **Los diagnósticos nacen mediante ConditionOnset y se registran mediante conditionStart().**
 - **Toda enfermedad pertenece a un Encounter concreto.**
 - **FHIR constituye únicamente una representación final de la información almacenada.**
 - **La separación entre Person y HealthRecord constituye una de las decisiones arquitectónicas más importantes del proyecto.**
-

Engineering Notes

Patrones identificados

- **Repository Pattern**
- **Event Log**
- **Single Responsibility Principle**
- **Temporal Data Model**

Idea fundamental del capítulo

HealthRecord no genera la historia clínica; la conserva. La historia clínica es la memoria completa de todo lo que el paciente ha vivido durante la simulación.

CAPÍTULO 9

Flujo Completo de un Evento Clínico: Del Módulo JSON al Recurso FHIR

9.1 Introducción

Hasta este momento hemos estudiado cada componente de Synthea de forma independiente.

Sin embargo, el verdadero funcionamiento del sistema únicamente puede comprenderse cuando se observa la interacción entre todos ellos.

El objetivo de este capítulo consiste en seguir el recorrido completo de un único evento clínico desde su definición dentro de un módulo JSON hasta su representación final como recurso FHIR.

Este recorrido constituye probablemente el concepto más importante de toda la arquitectura.

9.2 El escenario de estudio

Durante el análisis realizado sobre el código fuente oficial utilizaremos como ejemplo el diagnóstico de hipertensión.

El protocolo correspondiente se encuentra definido dentro del módulo.

modules/

hypertension.json

En dicho módulo encontramos el siguiente estado.

```
"Diagnose_Hypertension":{
```

```
"type":"ConditionOnset",
```

```
...
```

```
}
```

Este será el punto de partida de nuestro análisis.

9.3 Paso 1. El motor entra en el módulo

Cuando Generator comienza la simulación carga todos los módulos disponibles.

Uno de ellos será.

```
hypertension.json
```

El motor inicializa el módulo ejecutando.

```
Initial
```

Posteriormente continúa recorriendo las transiciones hasta alcanzar el estado.

```
ConditionOnset
```

En este momento todavía no existe ninguna enfermedad.

Únicamente existe una instrucción pendiente de ejecutar.

9.4 Paso 2. El motor interpreta el estado

Al encontrar.

```
"type":"ConditionOnset"
```

el motor no interpreta el nombre.

Interpreta el tipo.

Internamente realiza un proceso equivalente a.

ConditionOnset.diagnose(person,time)

Es importante observar que el motor todavía no crea ningún recurso clínico.

Simplemente ejecuta un método Java.

9.5 Paso 3. Modificación del objeto Person

El estado comienza modificando el paciente.

Por ejemplo.

Antes.

Person

Hipertensión = No

Después.

Person

Hipertensión = Sí

En este momento el paciente ya ha cambiado.

Sin embargo todavía no existe historia clínica.

9.6 Paso 4. Registro en HealthRecord

ConditionOnset solicita ahora a HealthRecord que registre el diagnóstico.

Durante el análisis del código encontramos la siguiente llamada.

conditionStart(...)

Este método realiza varias operaciones.

Crear una nueva Entry.

```
Entry condition = new Entry(...)
```

Localizar el Encounter actual.

```
Encounter encounter = currentEncounter(...)
```

Asociar el diagnóstico al Encounter.

```
encounter.conditions.add(condition)
```

Registrar la enfermedad activa.

```
present.put(...)
```

A partir de este momento la enfermedad forma parte oficialmente de la historia clínica.

9.7 Paso 5. Relación con Encounter

Uno de los aspectos más elegantes del diseño consiste en que las enfermedades nunca aparecen aisladas.

Siempre pertenecen a una consulta.

Podemos representarlo mediante el siguiente flujo.

Consulta

↓

Diagnóstico

↓

Historia clínica

Esto reproduce exactamente el funcionamiento de una historia clínica real.

9.8 Paso 6. Relación con Claims

En el sistema sanitario estadounidense.

El mismo diagnóstico genera automáticamente una nueva línea de facturación.

```
encounter.claim.addLineItem(...)
```

Por tanto.

Un único diagnóstico produce simultáneamente.

- historia clínica;
- facturación;
- costes;
- información para aseguradoras.

Esta decisión tendrá gran importancia cuando estudiemos la adaptación española.

9.9 Paso 7. Finalización de la simulación

Durante la simulación.

Toda la información permanece almacenada únicamente dentro de HealthRecord.

Todavía no existe ningún recurso FHIR.

9.10 Paso 8. Exportación

Cuando finaliza la simulación.

FHIRExporter comienza a recorrer HealthRecord.

Para cada Entry genera un recurso.

Por ejemplo.

Entry



Condition



FHIR Condition

El recurso contiene.

- código SNOMED;
- fecha;
- Encounter asociado;
- paciente;
- estado clínico.

Todos estos datos proceden de HealthRecord.

FHIR únicamente cambia su representación.

9.11 Flujo completo

Podemos resumir todo el proceso mediante el siguiente diagrama.

Generator



Person



hypertension.json



ConditionOnset

↓

diagnose()

↓

HealthRecord.conditionStart()

↓

Entry

↓

Encounter

↓

Claim

↓

FHIRExporter

↓

FHIR Condition

Este flujo constituye el recorrido completo de una enfermedad.

9.12 El recorrido inverso

Una de las técnicas más útiles para comprender Synthea consiste en recorrer el sistema en sentido contrario.

Partimos de un recurso FHIR.

FHIR Condition

Preguntamos.

¿Quién lo creó?

↓

HealthRecord.

¿Quién añadió la Entry?

↓

ConditionOnset.

¿Quién ejecutó ConditionOnset?

↓

El motor.

¿Quién ordenó ejecutarlo?

↓

El módulo.

¿Quién cargó el módulo?

↓

Generator.

Este procedimiento constituye la base del análisis de ingeniería inversa.

9.13 Aplicación práctica

Durante nuestras pruebas generamos un paciente.

Dentro del Bundle FHIR aparecieron recursos como.

- Patient.
- Encounter.
- Condition.
- DiagnosticReport.
- Claim.
- ExplanationOfBenefit.

Todos ellos proceden exactamente del mismo proceso descrito anteriormente.

No son recursos independientes.

Constituyen distintas representaciones del mismo episodio clínico.

9.14 Un único evento genera múltiples recursos

Este aspecto resulta especialmente importante.

Un único diagnóstico puede producir.

Condition

↓

Encounter

↓

DiagnosticReport

↓

Claim

↓

ExplanationOfBenefit

No existen cinco simulaciones distintas.

Existe un único evento clínico que posteriormente se representa mediante distintos recursos.

9.15 La importancia del desacoplamiento

Resulta tentador pensar que ConditionOnset debería crear directamente un recurso FHIR.

Sin embargo.

Esto rompería completamente la arquitectura.

Gracias al diseño actual.

ConditionOnset

↓

HealthRecord

↓

FHIR

CSV

JSON

CDA

El mismo evento puede exportarse simultáneamente a múltiples formatos.

9.16 Aplicación a la adaptación Python

Cuando recibamos la implementación desarrollada por el IRYCIS.

Este será probablemente el primer recorrido que deberemos reconstruir.

No bastará con localizar una clase llamada Patient.

Habrá que responder.

- ¿Cómo nace un diagnóstico?
- ¿Cómo se registra?
- ¿Dónde se almacena?
- ¿Cómo se exporta?
- ¿Continúa existiendo un flujo equivalente?

Responder a estas preguntas permitirá comparar ambas implementaciones con precisión.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **Un evento clínico atraviesa múltiples componentes antes de convertirse en un recurso FHIR.**
 - **El motor ejecuta estados, no genera directamente recursos clínicos.**
 - **HealthRecord constituye el punto central donde convergen todos los eventos.**
 - **FHIR representa únicamente la última etapa del proceso.**
 - **Un único diagnóstico puede producir múltiples recursos relacionados.**
 - **La técnica más útil para comprender Synthea consiste en recorrer el flujo tanto en sentido directo como inverso.**
-

Engineering Notes

Patrón arquitectónico identificado

Pipeline Architecture

La información atraviesa sucesivamente distintos componentes especializados.

Patrones presentes

- Event Pipeline
 - Repository Pattern
 - Interpreter Pattern
 - State Machine
 - Loose Coupling
-

Idea fundamental del capítulo

Un recurso FHIR no nace directamente. Es el resultado final de una cadena de transformaciones iniciada por un estado dentro de un módulo clínico.



Comentario del autor

Pablo, este capítulo representa exactamente el tipo de análisis que creo que harás en el IRYCIS.

No vas a preguntar:

"¿Dónde está la hipertensión?"

Vas a preguntar:

"¿Cómo viaja la información desde el protocolo clínico hasta el recurso FHIR en esta implementación?"

Ese cambio de perspectiva es el que convierte un análisis de código en un análisis de arquitectura.

CAPÍTULO 10

FHIR: Cómo Synthea Representa una Historia Clínica

10.1 Introducción

Una vez finalizada la simulación, el motor de Synthea dispone de una historia clínica completa almacenada en [HealthRecord](#).

Sin embargo, dicha historia únicamente existe como una representación interna del sistema.

Para que esta información pueda utilizarse por aplicaciones externas resulta necesario transformarla a un formato estándar.

Synthea utiliza para ello **FHIR (Fast Healthcare Interoperability Resources)**, el estándar desarrollado por HL7 para el intercambio de información clínica.

Es importante comprender que:

FHIR no participa en la simulación.

FHIR únicamente representa el resultado de dicha simulación.

10.2 ¿Qué es FHIR?

FHIR (Fast Healthcare Interoperability Resources) es un estándar internacional cuyo objetivo consiste en permitir que diferentes sistemas sanitarios intercambien información clínica utilizando una estructura común.

En lugar de almacenar toda la historia clínica en un único documento, FHIR divide la información en pequeñas unidades independientes llamadas **Resources**.

Cada recurso representa un único concepto clínico.

Por ejemplo.

Recurso	Representa
Patient	Paciente
Encounter	Consulta
Condition	Diagnóstico
Observation	Observación clínica
Procedure	Procedimiento
MedicationRequest	Prescripción
Organization	Hospital
Practitioner	Profesional sanitario
Claim	Facturación
ExplanationOfBenefit	Resultado económico del episodio

Por tanto.

Una historia clínica completa no es un único recurso FHIR.

Es un conjunto de cientos o miles de recursos relacionados entre sí.

10.3 Bundle: la historia clínica completa

Durante nuestras pruebas generamos el archivo:

Burl285_Lubowitz58_....json

Al abrirlo observamos:

```
{  
  "resourceType": "Bundle",  
  
  "type": "transaction"
```

}

Este detalle es fundamental.

El archivo no representa un paciente.

Representa un **Bundle**.

Un Bundle es simplemente un contenedor de recursos FHIR.

Podemos imaginarlo como una carpeta.

Bundle

|

|— Patient

|— Encounter

|— Condition

|— Procedure

|— MedicationRequest

|— DiagnosticReport

|— Claim

|— Organization

|— Practitioner

El Bundle constituye la historia clínica completa.

10.4 El recurso Patient

Todo Bundle comienza con un recurso Patient.

Ejemplo obtenido durante nuestro análisis.

"resourceType":"Patient"

Este recurso contiene exclusivamente información demográfica.

Por ejemplo.

- nombre;
- sexo;
- fecha de nacimiento;
- dirección;
- identificadores;
- estado civil.

Es importante destacar que **Patient no contiene enfermedades**.

Tampoco contiene consultas.

Ni medicación.

Patient representa únicamente la identidad del individuo.

10.5 El recurso Encounter

Uno de los recursos más importantes es Encounter.

Durante el análisis observamos múltiples consultas.

Ejemplo.

"resourceType":"Encounter"

Cada Encounter representa una interacción entre el paciente y el sistema sanitario.

Puede corresponder a:

- Atención Primaria.
- Consulta especializada.
- Urgencias.
- Hospitalización.

Toda actividad clínica ocurre dentro de un Encounter.

10.6 El recurso Condition

Condition representa un diagnóstico.

Ejemplo.

"resourceType":"Condition"

Cada recurso contiene.

- código SNOMED.
- estado clínico.
- fecha de inicio.
- Encounter asociado.
- paciente.

Obsérvese que el diagnóstico mantiene una referencia al Encounter donde fue realizado.

Esto reproduce exactamente el comportamiento observado previamente en HealthRecord.

10.7 El recurso Observation

Observation representa cualquier medición clínica.

Ejemplos.

- tensión arterial.
- frecuencia cardíaca.
- glucemia.
- peso.
- altura.
- resultados analíticos.

La mayoría de constantes vitales aparecen representadas mediante este recurso.

10.8 El recurso Procedure

Procedure representa cualquier intervención realizada sobre el paciente.

Ejemplos.

- cirugía;
- biopsia;
- radiografía;
- vacunación;
- exploraciones.

Cada procedimiento queda vinculado a un Encounter concreto.

10.9 El recurso MedicationRequest

MedicationRequest representa la prescripción de un medicamento.

No representa el medicamento en sí.

Representa la decisión clínica de prescribirlo.

Este detalle resulta muy importante.

Ejemplo.

MedicationOrder

↓

MedicationRequest

El estado MedicationOrder observado durante el análisis del motor termina convirtiéndose en este recurso.

10.10 Practitioner y Organization

Durante nuestras pruebas también se generaron dos archivos adicionales.

practitionerInformation.json

y

hospitalInformation.json

Estos archivos contienen recursos independientes.

Practitioner

Representa profesionales sanitarios.

Por ejemplo.

- médicos;
 - especialistas;
 - enfermería.
-

Organization

Representa instituciones sanitarias.

Por ejemplo.

- hospitales;
- centros de salud;
- clínicas.

Separar esta información del Bundle del paciente reduce duplicidades.

Un mismo profesional puede atender a cientos de pacientes.

10.11 Claim y ExplanationOfBenefit

Uno de los aspectos más característicos del Synthea estadounidense consiste en la presencia de recursos económicos.

Claim

Representa la factura generada durante una asistencia sanitaria.

Incluye.

- procedimientos;
 - diagnósticos;
 - costes.
-

ExplanationOfBenefit

Representa el resultado económico tras aplicar el seguro sanitario.

Puede incluir.

- cantidad pagada;
- aseguradora;
- copago;
- importe restante.

Estos recursos reflejan el funcionamiento del sistema sanitario estadounidense.

10.12 Relaciones entre recursos

FHIR no funciona mediante jerarquías.

Funciona mediante referencias.

Ejemplo.

Patient

↓

Encounter

↓

Condition

Condition no contiene toda la información del paciente.

Simplemente mantiene una referencia.

subject

↓

Patient

Del mismo modo.

Condition

↓

Encounter

mantiene otra referencia.

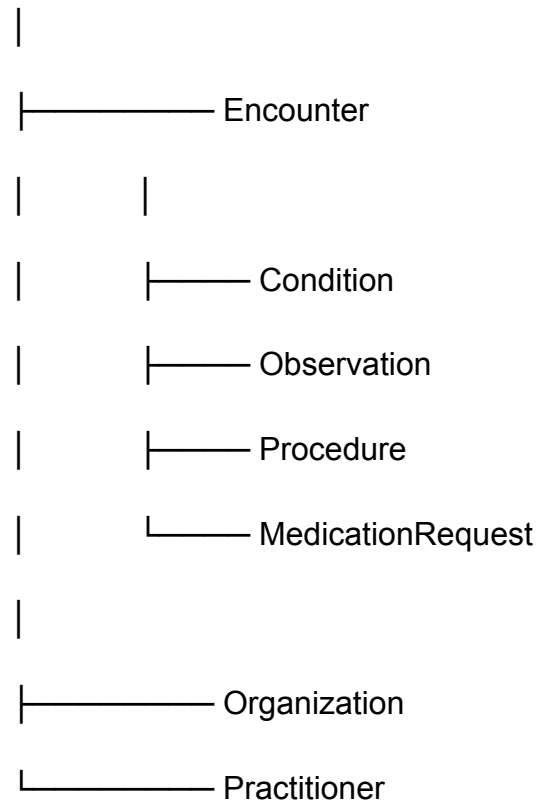
Esta estructura evita duplicidades.

10.13 El Bundle como grafo

Desde un punto de vista arquitectónico.

Un Bundle puede entenderse como un grafo.

Patient



No existe una jerarquía rígida.

Existe una red de recursos relacionados.

10.14 Correspondencia con la arquitectura de Synthea

Todo recurso FHIR posee un origen muy concreto dentro del motor.

Motor	FHIR
Person	Patient
Encounter	Encounter
ConditionOnset	Condition
Observation	Observation
Procedure	Procedure
MedicationOrder	MedicationRequest
Provider	Organization
Clinician	Practitioner

Esta tabla será extremadamente útil durante el análisis de la implementación Python.

10.15 Lo que NO debemos hacer

Uno de los errores más frecuentes consiste en intentar comprender Synthea leyendo únicamente los recursos FHIR.

Esto resulta insuficiente.

FHIR representa únicamente el resultado final.

La verdadera lógica reside en.

Modules

↓

States

↓

Person

↓

HealthRecord

FHIR únicamente transforma dicha información.

10.16 Implicaciones para la adaptación española

Este capítulo proporciona una pista muy importante.

FHIR es un estándar internacional.

Por tanto.

No esperamos grandes modificaciones en los recursos FHIR.

Lo que probablemente cambie será el contenido.

Ejemplos.

En Estados Unidos.

Claim

↓

Humana

En España podría aparecer.

Servicio Madrileño de Salud

↓

SNS



Mutuas

Pero el recurso FHIR seguirá siendo Claim (o el mecanismo que decidan utilizar).

Por tanto.

Esperamos cambios en la información.

No necesariamente en el estándar.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **FHIR representa el resultado de la simulación, no la simulación en sí misma.**
 - **Una historia clínica se representa mediante un Bundle compuesto por múltiples recursos.**
 - **Cada recurso posee una correspondencia directa con elementos internos del motor.**
 - **FHIR utiliza referencias entre recursos para evitar duplicidades.**
 - **Los recursos económicos reflejan el funcionamiento del sistema sanitario estadounidense.**
 - **La adaptación española probablemente modificará el contenido de dichos recursos, pero no el estándar FHIR.**
-



Engineering Notes

Patrón arquitectónico identificado

Graph-based Information Model

FHIR representa la información como una red de recursos relacionados mediante referencias.

Principios observados

- Normalización de datos
 - Separación de entidades
 - Reutilización de recursos
 - Interoperabilidad
-

Idea fundamental del capítulo

FHIR no explica cómo se ha generado una historia clínica.
Únicamente proporciona un lenguaje estándar para representarla e intercambiarla entre sistemas.

Comentario del autor (el detalle que creo que más te ayudará en el IRYCIS)

Pablo, aquí hay una idea que creo que va a darte mucha ventaja.

Cuando recibas la versión Python, **no empieces mirando los recursos FHIR.**

Empieza preguntándote:

¿Cómo han llegado estos recursos hasta aquí?

Porque si los recursos FHIR son prácticamente iguales pero el recorrido interno ha cambiado, significará que el equipo ha modificado la arquitectura del simulador sin alterar el estándar de interoperabilidad.

Eso es exactamente el tipo de análisis que, en mi opinión, esperan de alguien que va a comparar ambas implementaciones.

CAPÍTULO 11

Ingeniería Inversa de un Paciente Generado

11.1 Introducción

Hasta este momento hemos estudiado el funcionamiento interno del motor de simulación y la arquitectura general de Synthea.

Sin embargo, el verdadero conocimiento de un sistema software aparece cuando se es capaz de reconstruir el recorrido completo seguido por un caso real.

El objetivo de este capítulo consiste en aplicar todos los conceptos estudiados anteriormente sobre un paciente generado durante nuestras propias pruebas.

En lugar de estudiar únicamente el código fuente, analizaremos el resultado final producido por el sistema y reconstruiremos todo el proceso interno que ha llevado a generar dicha historia clínica.

Este procedimiento constituye una técnica clásica de ingeniería inversa y será la misma metodología que posteriormente utilizaremos para estudiar la adaptación desarrollada por el IRYCIS.

11.2 El paciente analizado

Durante la ejecución de Synthea se generó un único paciente sintético.

Identificador.

df60c672-df51-fc7f-0a36-65d4e70a2833

Nombre.

Mr. Burl285 Lubowitz58

Sexo.

Masculino.

Fecha de nacimiento.

16 de diciembre de 1973.

Lugar de residencia.

Braintree, Massachusetts.

Estado civil.

Soltero.

Estos datos proceden directamente del recurso Patient generado por Synthea.

11.3 Primera observación

Lo primero que debe comprenderse es que este archivo **no representa un paciente**, sino una historia clínica completa.

El fichero generado corresponde a un Bundle FHIR.

Esto significa que contiene múltiples recursos relacionados entre sí.

Entre ellos encontramos.

- Patient
- Encounter
- Condition
- DiagnosticReport
- Claim
- ExplanationOfBenefit
- Practitioner
- Organization

Cada uno representa un aspecto distinto del mismo episodio clínico.

11.4 Reconstrucción cronológica

Una forma muy útil de estudiar un paciente consiste en reconstruir cronológicamente su evolución.

A partir del Bundle generado obtenemos el siguiente resumen.

Nacimiento

16 de diciembre de 1973

Se crea el recurso Patient.

Todavía no existen consultas.

No existen enfermedades.

No existe historia clínica.

Únicamente existe el objeto Person inicializado por Generator.

Primera consulta registrada

9 de febrero de 1992

Edad aproximada.

18 años.

Tipo de Encounter.

General examination of patient.

Profesional.

Dr. Sherwood961 O'Conner199.

Centro sanitario.

HEALTHCARE SOUTH PC.

Este evento corresponde probablemente al módulo Wellness, encargado de las revisiones periódicas.

Primer diagnóstico

Durante esa misma consulta aparece la condición.

Received higher education (finding)

Código SNOMED.

224299000

Desde el punto de vista arquitectónico este diagnóstico ha seguido exactamente el recorrido estudiado anteriormente.

Wellness Module

↓

ConditionOnset

↓

diagnose()

↓

HealthRecord.conditionStart()

↓

FHIR Condition

Segundo episodio clínico

12 de mayo de 1992

Nueva consulta.

Tipo.

Encounter for symptom.

Durante ella aparece una nueva condición.

Chronic sinusitis

Código SNOMED.

40055000

Además observamos que Synthea genera automáticamente.

- Encounter
- Condition
- DiagnosticReport
- Claim
- ExplanationOfBenefit

Todo ello asociado al mismo episodio clínico.

Episodio de obesidad

24 de febrero de 2002

Durante una revisión general aparece el diagnóstico.

Body mass index 30+ - obesity

Código SNOMED.

162864005

Este episodio constituye un excelente ejemplo de cómo una enfermedad puede aparecer muchos años después del nacimiento del paciente, respetando una evolución temporal coherente.

11.5 Reconstrucción arquitectónica

Vamos a analizar únicamente el episodio de obesidad.

El recorrido completo ha sido.

Generator

↓

Person

↓

Wellness Module

↓

ConditionOnset

↓

diagnose()

↓

HealthRecord

↓

Condition

↓

Encounter

↓

Claim

↓

ExplanationOfBenefit

↓

FHIR Bundle

Obsérvese que el recurso FHIR representa únicamente el último paso.

11.6 Relaciones entre recursos

Uno de los aspectos más interesantes del Bundle consiste en que todos los recursos mantienen referencias cruzadas.

Por ejemplo.

El recurso Condition mantiene una referencia al Encounter donde fue diagnosticado.

El Encounter mantiene una referencia al Practitioner responsable.

El Practitioner mantiene una referencia a la Organization correspondiente.

Finalmente el Claim reutiliza tanto el Encounter como el Condition.

Podemos representarlo así.

Patient

↓

Encounter

↓

Condition

↓

Claim

↓

ExplanationOfBenefit

Todos pertenecen al mismo episodio asistencial.

11.7 Lo que el paciente nos permite deducir

Sin necesidad de leer el código fuente podemos inferir múltiples aspectos sobre la arquitectura.

Por ejemplo.

Existe un sistema de revisiones periódicas

Las consultas generales aparecen separadas por varios años.

Esto sugiere la existencia de un módulo específico encargado de los controles preventivos.

Existe un sistema de generación automática de documentación

Cada consulta genera automáticamente.

- DiagnosticReport.
- DocumentReference.

Esto indica que el motor produce documentación clínica además de diagnósticos.

Existe un sistema económico

Cada episodio genera.

- Claim.
- ExplanationOfBenefit.

Lo que confirma que la simulación incluye aspectos financieros propios del sistema sanitario estadounidense.

11.8 ¿Qué no vemos?

También es importante analizar lo que no aparece.

En este paciente no observamos.

- hipertensión;
- diabetes;
- medicación crónica.

Esto demuestra que los módulos no actúan siempre.

Su ejecución depende de condiciones y probabilidades.

11.9 Ingeniería inversa

A partir del Bundle podemos reconstruir el flujo interno seguido por el motor.

Ejemplo.

FHIR Condition

↑

HealthRecord

↑

ConditionOnset

↑

Wellness Module

↑

Generator

Este recorrido inverso constituye la herramienta más útil para comprender cualquier implementación de Synthea.

11.10 Preparación para la adaptación Python

Este mismo procedimiento será el que utilizaremos cuando recibamos el código del IRYCIS.

No comenzaremos leyendo clases Python.

Comenzaremos generando un paciente.

Posteriormente responderemos.

- ¿Qué recursos aparecen?
- ¿Qué recursos desaparecen?
- ¿Qué cambia?
- ¿Qué permanece igual?
- ¿Qué recorrido interno explica dichas diferencias?

De esta forma podremos comparar ambas implementaciones independientemente del lenguaje de programación utilizado.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **La mejor forma de estudiar Synthea consiste en realizar ingeniería inversa sobre pacientes generados.**
 - **Un Bundle FHIR representa una historia clínica completa y no un único recurso.**
 - **Cada episodio asistencial genera múltiples recursos relacionados.**
 - **El recorrido desde el módulo hasta FHIR puede reconstruirse completamente.**
 - **El análisis de pacientes reales constituye la metodología que posteriormente aplicaremos sobre la versión Python del IRYCIS.**
-

Engineering Notes

Técnica utilizada

Reverse Engineering

En lugar de comenzar por el código fuente, el análisis comienza por los resultados generados y reconstruye el recorrido interno seguido por el sistema.

Beneficios

- Permite validar la arquitectura.
 - Facilita la comprensión del motor.
 - Hace posible comparar implementaciones distintas.
 - Reduce enormemente el tiempo necesario para comprender nuevos repositorios.
-

Idea fundamental del capítulo

La historia clínica generada por Synthea puede utilizarse como un mapa para reconstruir el funcionamiento interno completo del motor de simulación.



Comentario del autor

Pablo, aquí creo que está el mayor cambio respecto a cómo suele estudiarse un proyecto de software.

La mayoría de personas empiezan leyendo clases.

Yo creo que, para este proyecto, **es mucho más potente empezar leyendo pacientes.**

Cuando recibas la versión Python, mi recomendación será exactamente la misma:

1. Generar un paciente.
2. Leer su historia clínica.
3. Identificar los recursos producidos.
4. Reconstruir el recorrido interno.
5. Solo entonces abrir el código.

Si haces eso, comprenderás la implementación mucho más rápido que empezando por cientos de archivos Python.

CAPÍTULO 12

Patrones de Diseño y Principios de Ingeniería Utilizados por Synthea

12.1 Introducción

Hasta este momento hemos analizado la arquitectura funcional de Synthea.

Sin embargo, todavía no hemos estudiado una cuestión fundamental.

¿Por qué el código está organizado exactamente de esta forma?

La respuesta reside en la utilización de numerosos patrones clásicos de ingeniería del software.

Lejos de ser una colección de clases Java, Synthea constituye un excelente ejemplo de aplicación práctica de múltiples principios SOLID y patrones GoF (Gang of Four).

Comprender estos patrones permitirá interpretar mucho mejor tanto la implementación original como cualquier adaptación posterior.

12.2 Principio de Responsabilidad Única (SRP)

El principio de responsabilidad única establece que una clase debe tener un único motivo para cambiar.

Synthea aplica este principio prácticamente en toda su arquitectura.

Ejemplos.

Clase	Responsabilidad
-------	-----------------

Generator	Coordinar la simulación
Person	Representar el paciente
Module	Representar un protocolo clínico
State	Ejecutar una acción clínica
HealthRecord	Registrar eventos
FHIRExporter	Exportar recursos FHIR

Cada componente realiza una única tarea.

Nunca encontramos clases que mezclen responsabilidades.

Ejemplo

ConditionOnset.

Su única responsabilidad consiste en crear un diagnóstico.

No exporta FHIR.

No crea consultas.

No calcula costes.

Únicamente diagnostica.

Esto facilita enormemente el mantenimiento.

12.3 Open / Closed Principle

Este principio establece que un sistema debe estar abierto a la extensión pero cerrado a la modificación.

Synthea constituye un ejemplo excelente.

Supongamos que deseamos añadir una nueva enfermedad.

No modificamos Generator.

No modificamos Person.

No modificamos HealthRecord.

Simplemente añadimos.

modules/

new_disease.json

El motor continúa funcionando exactamente igual.

Consecuencia

El motor permanece estable.

Los protocolos evolucionan independientemente.

12.4 State Pattern

Probablemente el patrón más importante de todo el proyecto.

Cada protocolo clínico se implementa mediante una máquina de estados.

Initial

↓

Encounter

↓

Observation

↓

ConditionOnset

↓

MedicationOrder

↓

Terminal

Cada estado representa una acción independiente.

El motor únicamente los ejecuta.

Ventajas

- protocolos legibles;
 - fácil mantenimiento;
 - reutilización;
 - modularidad.
-

12.5 Interpreter Pattern

Este patrón aparece constantemente en Synthea.

El motor no contiene enfermedades.

Contiene un intérprete.

Por ejemplo.

El JSON dice.

```
"type":"ConditionOnset"
```

El motor interpreta.

```
ConditionOnset.diagnose(...)
```

Es decir.

El JSON constituye un pequeño lenguaje específico.

El motor interpreta dicho lenguaje.

Consecuencia

Modificar el comportamiento clínico no requiere modificar Java.

12.6 Repository Pattern

HealthRecord constituye un ejemplo prácticamente perfecto de Repository Pattern.

Su función consiste en almacenar información.

No genera.

No calcula.

No interpreta.

Simplemente conserva.

Podemos resumirlo así.

Motor

↓

HealthRecord

↓

FHIR

12.7 Data Driven Architecture

Este patrón constituye probablemente la característica más distintiva de Synthea.

En la mayoría de aplicaciones el comportamiento reside en el código.

En Synthea.

El comportamiento reside en los datos.

Concretamente.

hypertension.json

diabetes.json

pregnancy.json

El motor únicamente ejecuta dichos datos.

Esto convierte a Synthea en una arquitectura dirigida por datos.

12.8 Loose Coupling

Los componentes apenas conocen detalles internos unos de otros.

Por ejemplo.

ConditionOnset desconoce FHIR.

FHIRExporter desconoce ConditionOnset.

HealthRecord desconoce Generator.

Cada componente interactúa únicamente mediante interfaces bien definidas.

Beneficios

- mayor reutilización;
 - menor dependencia;
 - facilidad para sustituir componentes.
-

12.9 High Cohesion

Cada componente agrupa funcionalidades muy relacionadas.

Ejemplo.

HealthRecord contiene únicamente elementos relacionados con la historia clínica.

Exporter contiene únicamente lógica de exportación.

Module contiene únicamente protocolos.

Esta alta cohesión facilita enormemente la comprensión del código.

12.10 Single Source of Truth

Toda la información del paciente reside en un único objeto.

Person

Todos los módulos trabajan sobre él.

Nunca existen copias.

Nunca existen versiones alternativas.

Esto evita inconsistencias.

12.11 Event Driven Architecture

La simulación se construye mediante eventos.

Ejemplos.

ConditionOnset

↓

Nuevo diagnóstico

MedicationOrder

↓

Nueva prescripción
Procedure

↓

Nuevo procedimiento

Cada evento modifica el estado del paciente y amplía la historia clínica.

12.12 Pipeline Pattern

Durante el capítulo anterior estudiamos el recorrido completo de un evento.

Module

↓

State

↓

Person

↓

HealthRecord

↓

FHIRExporter

↓

FHIR

Este flujo constituye un ejemplo clásico de arquitectura en pipeline.

Cada componente transforma parcialmente la información.

12.13 Patrones que NO aparecen

También resulta interesante observar qué patrones no parecen utilizarse.

Por ejemplo.

No encontramos una arquitectura basada en microservicios.

No encontramos programación reactiva.

No encontramos Event Sourcing completo.

No encontramos CQRS.

La arquitectura permanece relativamente sencilla.

Esto facilita enormemente su mantenimiento.

12.14 Calidad arquitectónica

Tras analizar el código fuente podemos destacar varios aspectos especialmente positivos.

Separación clara de responsabilidades.

Bajo acoplamiento.

Alta cohesión.

Excelente reutilización.

Facilidad para añadir nuevos protocolos.

Motor completamente independiente del conocimiento clínico.

Desde un punto de vista académico constituye un proyecto muy bien diseñado.

12.15 Aspectos mejorables

No obstante también pueden identificarse ciertas limitaciones.

Los módulos JSON resultan poco expresivos para protocolos extremadamente complejos.

Algunos estados concentran bastante lógica.

La incorporación de nuevos tipos de estado requiere modificar el motor.

La depuración de protocolos largos puede resultar complicada.

12.16 Implicaciones para el IRYCIS

Este capítulo proporciona una herramienta especialmente útil para analizar la adaptación Python.

Cuando recibamos el código no deberemos limitarnos a comparar clases.

Deberemos responder cuestiones como.

- ¿Se mantiene el State Pattern?
- ¿Continúa existiendo una arquitectura dirigida por datos?
- ¿HealthRecord sigue funcionando como repositorio?
- ¿Person continúa siendo la única fuente de verdad?
- ¿Se mantienen los principios SOLID?

Responder a estas preguntas permitirá evaluar la calidad arquitectónica de la adaptación.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **Synthea constituye un excelente ejemplo práctico de múltiples patrones clásicos de ingeniería del software.**
 - **La separación entre motor y módulos representa una arquitectura dirigida por datos.**
 - **El motor implementa un Interpreter Pattern.**
 - **Los módulos implementan una State Machine.**
 - **HealthRecord actúa como Repository.**
 - **Person constituye la única fuente de verdad del estado del paciente.**
 - **Toda la arquitectura sigue de forma consistente los principios SOLID.**
-

Engineering Notes

Patrones identificados

Patrón	Dónde aparece
Single Responsibility	Toda la arquitectura
Open / Closed	Módulos JSON
State Pattern	States
Interpreter Pattern	Motor
Repository Pattern	HealthRecord
Data Driven Design	Modules
Pipeline	Flujo completo
Loose Coupling	Relaciones entre componentes
High Cohesion	Organización de clases
Single Source of Truth	Person

La arquitectura de Synthea en una frase

Synthea es un motor de simulación clínica dirigido por datos que interpreta protocolos médicos descritos mediante máquinas de estados y construye historias clínicas longitudinales utilizando una arquitectura modular basada en principios SOLID y patrones clásicos de ingeniería del software.

Comentario del autor (probablemente el más importante del libro)

Pablo, creo que aquí está la diferencia entre "leer código" y "hacer ingeniería".

Cuando abras la versión Python, no quiero que pienses:

"Aquí hay un `patient.py`."

Quiero que pienses:

"¿Dónde está el Repository? ¿Dónde está el State Pattern? ¿Dónde está el Interpreter? ¿Han mantenido la arquitectura dirigida por datos?"

Eso cambia completamente la forma de analizar un proyecto.

Ya no comparas archivos.

Comparas decisiones de diseño.

Y eso es exactamente lo que, por todo lo que me has contado, creo que esperan de ti en el IRYCIS.

CAPÍTULO 13

Adaptación de Synthea al Sistema Sanitario Español

13.1 Introducción

Synthea fue desarrollado originalmente por **The MITRE Corporation** con el objetivo de simular el funcionamiento del sistema sanitario estadounidense.

Como consecuencia, numerosos componentes de su arquitectura reflejan características propias de dicho sistema.

Entre ellas.

- seguros privados;
- aseguradoras;
- facturación;
- copagos;
- proveedores sanitarios estadounidenses;
- modelos de financiación.

Por tanto.

Una adaptación al sistema sanitario español no consiste simplemente en traducir el código.

Consiste en modificar la simulación para representar correctamente un modelo asistencial completamente distinto.

13.2 ¿Qué debería mantenerse?

Uno de los errores más frecuentes sería intentar reescribir completamente Synthea.

Desde un punto de vista arquitectónico esto tendría poco sentido.

Gran parte del diseño original resulta completamente independiente del país.

Por tanto.

La filosofía general debería mantenerse.

Entre los componentes que probablemente deberían conservarse encontramos.

El motor de simulación

El motor únicamente interpreta módulos.

No contiene conocimiento clínico.

No depende del sistema sanitario.

Por tanto.

No existe ninguna razón arquitectónica evidente para modificar su filosofía.

Person

El concepto de paciente es universal.

Todo sistema sanitario necesita representar.

- edad;
- sexo;
- enfermedades;
- atributos;
- evolución temporal.

Por tanto.

La existencia de un objeto central equivalente a Person debería mantenerse.

HealthRecord

La historia clínica también constituye un concepto universal.

Independientemente del país.

Siempre existirán.

- consultas;
- diagnósticos;
- procedimientos;
- tratamientos.

Por tanto.

HealthRecord debería mantenerse prácticamente sin cambios.

Máquina de estados

Los protocolos clínicos continúan representándose perfectamente mediante máquinas de estados.

Por tanto.

No parece existir ninguna razón para abandonar este paradigma.

13.3 ¿Qué debería cambiar?

Aquí comienzan las diferencias importantes.

Providers

En la versión estadounidense aparecen.

- hospitales;
- clínicas;
- nursing homes;
- urgent care.

España posee una organización distinta.

Por ejemplo.

- Centros de Salud.
- Atención Primaria.
- Hospitales públicos.
- Hospitales privados.
- Mutuas.
- Atención domiciliaria.

Por tanto.

La representación de proveedores sanitarios debería adaptarse completamente.

Sistema de aseguramiento

Estados Unidos.

Paciente

↓

Seguro privado

↓

Hospital

↓

Claim

↓

Insurance

↓

ExplanationOfBenefit

España.

Paciente

↓

SNS



Centro de Salud



Derivación



Hospital



Alta

La diferencia arquitectónica es enorme.

Costes

En Synthea observamos recursos como.

Claim



ExplanationOfBenefit

Estos recursos reflejan la existencia de aseguradoras.

En España.

La financiación es diferente.

Probablemente habría que modelar.

- costes para el sistema;
- costes hospitalarios;
- costes farmacéuticos;
- financiación pública.

No necesariamente facturación individual.

13.4 Atención Primaria

Probablemente la modificación más importante.

En el modelo estadounidense.

Muchos procesos comienzan directamente en consultas especializadas.

En España.

La puerta de entrada suele ser Atención Primaria.

Por ejemplo.

En lugar de.

Initial

↓

ConditionOnset

Podría aparecer.

Initial

↓

Primary Care Encounter

↓

Evaluación

↓

Derivación

↓

ConditionOnset

Este cambio modifica el flujo clínico.

No el motor.

13.5 Derivaciones

España presenta un sistema mucho más jerarquizado.

Podría ser necesario introducir estados nuevos.

Por ejemplo.

Referral

o.

PrimaryCareEvaluation

Si dichos estados no existen actualmente.

El motor debería aprender a interpretarlos.

Este constituye uno de los posibles motivos para modificar el motor.

13.6 Medicación

Los medicamentos utilizados por Synthea representan la práctica clínica estadounidense.

Una adaptación española debería revisar.

- principios activos;
- guías clínicas;
- financiación;
- disponibilidad.

No necesariamente cambiar la arquitectura.

Pero sí el contenido clínico.

13.7 Guías clínicas

Durante nuestro análisis observamos que los módulos citan referencias como.

- ACC/AHA.
- ESC.
- Canadian Guidelines.

Una adaptación española probablemente incorporaría.

- Ministerio de Sanidad.
- Guías SEMFYC.
- Guías SEMI.
- Guías SEC.
- Protocolos autonómicos.

Los módulos continuarían siendo JSON.

Cambiaría únicamente el conocimiento clínico.

13.8 ¿Basta con modificar los módulos?

Esta es probablemente la pregunta más importante de todo el proyecto.

Después de analizar Synthea podemos responder.

Depende del tipo de adaptación.

Caso 1

Cambiar protocolos clínicos.

Sí.

Basta modificar los módulos.

Caso 2

Cambiar probabilidades.

Sí.

Basta modificar los módulos.

Caso 3

Cambiar medicamentos.

Sí.

Basta modificar los módulos.

Caso 4

Cambiar el modelo asistencial.

Depende.

Si puede representarse mediante estados existentes.

Sí.

Si requiere nuevos tipos de estado.

No.

Habrà que modificar el motor.

Caso 5

Cambiar el sistema económico.

Probablemente requerirá modificar tanto módulos como componentes relacionados con costes.

13.9 Hipótesis sobre la adaptación Python

Antes de disponer del repositorio del IRYCIS podemos formular varias hipótesis.

Hipótesis 1

La filosofía general de simulación se mantiene.

Hipótesis 2

Person continúa existiendo bajo otro nombre.

Hipótesis 3

HealthRecord permanece prácticamente igual.

Hipótesis 4

Los mayores cambios aparecen en Providers, costes y módulos clínicos.

Hipótesis 5

Python facilita la integración con herramientas de investigación como pandas, NumPy o scikit-learn.

Hipótesis 6

La adaptación introduce componentes específicos del Sistema Nacional de Salud.

13.10 Riesgos de una adaptación

Modificar excesivamente el motor puede romper la compatibilidad con el proyecto original.

Modificar únicamente los módulos puede resultar insuficiente para representar correctamente el sistema sanitario español.

Por tanto.

Uno de los objetivos principales de la adaptación consiste en encontrar un equilibrio entre reutilización y especialización.

13.11 Criterios para evaluar la adaptación

Durante el análisis del código Python deberíamos responder.

Arquitectura

- ¿Se mantiene el motor?
 - ¿Se mantienen los módulos?
 - ¿Se conserva la máquina de estados?
-

Modelo clínico

- ¿Se representan correctamente los flujos españoles?
 - ¿Existe Atención Primaria?
 - ¿Existen derivaciones?
 - ¿Cómo se representan los hospitales públicos?
-

Economía

- ¿Cómo se representan los costes?
 - ¿Qué sustituye a Claims?
 - ¿Qué sustituye a ExplanationOfBenefit?
-

Calidad del software

- ¿Se mantienen los principios SOLID?
 - ¿La arquitectura continúa desacoplada?
-

13.12 Conclusión

Una adaptación nacional de Synthea no consiste únicamente en cambiar nombres o traducir texto.

Consiste en adaptar un modelo de simulación clínica a un sistema sanitario completamente diferente.

El reto consiste en introducir dichas modificaciones manteniendo las fortalezas arquitectónicas del proyecto original.

Desde el punto de vista de la ingeniería del software.

La mejor adaptación será aquella que modifique exclusivamente aquellos componentes cuya responsabilidad dependa realmente del contexto sanitario.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- **Gran parte de la arquitectura original debería mantenerse.**
- **Los mayores cambios deberían concentrarse en módulos, proveedores, costes y flujos asistenciales.**
- **No todas las adaptaciones requieren modificar el motor.**

- La adaptación española debe representar un modelo asistencial diferente sin perder la filosofía arquitectónica original.
 - La comparación entre Java y Python deberá centrarse en responsabilidades y no únicamente en nombres de clases.
-

Engineering Notes

Pregunta clave del proyecto

¿Qué problema concreto no podía resolverse modificando únicamente los módulos JSON y justificó el desarrollo de una adaptación en Python?

Esta será probablemente la pregunta más importante de toda la investigación.

Hipótesis principal

Si la adaptación Python modifica el motor, debería existir una justificación arquitectónica clara. Si únicamente adapta protocolos clínicos, probablemente habría bastado con modificar los módulos.

Idea fundamental del capítulo

El verdadero reto no consiste en traducir Synthea a Python. Consiste en decidir qué debe permanecer igual y qué debe cambiar para representar correctamente el sistema sanitario español.

CAPÍTULO 14

Metodología Profesional para Analizar la Adaptación Python del IRYCIS

14.1 Introducción

Una vez comprendida la arquitectura de Synthea surge una nueva pregunta.

¿Cómo debe analizarse correctamente una adaptación desarrollada por un tercero?

La respuesta no consiste en comenzar leyendo código.

Ese enfoque suele conducir rápidamente a la desorientación.

Repositorios de gran tamaño pueden contener cientos de archivos y miles de líneas de código.

Leerlos secuencialmente rara vez permite comprender el funcionamiento global del sistema.

Por este motivo, el análisis arquitectónico debe realizarse siguiendo un procedimiento sistemático.

El objetivo de este capítulo consiste en definir dicha metodología.

14.2 Objetivo del análisis

El objetivo **no** consiste en aprender Python.

El objetivo **no** consiste en traducir Java.

El objetivo consiste en responder una única pregunta.

¿Cómo ha cambiado la arquitectura respecto al proyecto original de Synthea?

Todo el análisis debe orientarse hacia esa cuestión.

14.3 Qué NO hacer

Al recibir el repositorio Python resulta tentador abrir inmediatamente el archivo principal.

Por ejemplo.

`main.py`

o

`patient.py`

Este enfoque suele producir una falsa sensación de comprensión.

El lector entiende pequeños fragmentos de código, pero desconoce completamente la arquitectura.

Por tanto.

No debe comenzarse leyendo código.

14.4 El orden correcto de análisis

El análisis debería seguir exactamente el siguiente orden.

README

↓

Estructura del proyecto

↓

Arquitectura

↓

Motor

↓

Paciente

↓

Historia clínica

↓

Módulos

↓

Exportación

↓

Detalles de implementación

Este orden reduce enormemente el tiempo necesario para comprender el sistema.

14.5 Paso 1. Leer el README

El README constituye el documento de mayor valor arquitectónico del proyecto.

Su lectura debe responder preguntas como.

- ¿Cuál es el objetivo del proyecto?
- ¿Qué componentes existen?
- ¿Qué diferencias respecto a Synthea se mencionan?
- ¿Cómo se ejecuta?
- ¿Qué dependencias utiliza?

No debe utilizarse únicamente como guía de instalación.

Debe entenderse como un documento de diseño.

14.6 Paso 2. Analizar la estructura del repositorio

Antes de abrir cualquier archivo conviene construir un mapa del proyecto.

Ejemplo.

python-synthea/

```
|
|
|— engine/
|
|— patient/
|
|— modules/
|
|— exporters/
|
|— providers/
|
|— costs/
|
|— config/
|
|— main.py
```

Todavía no interesa el contenido.

Interesa únicamente responder.

¿Qué responsabilidades parecen existir?

14.7 Paso 3. Localizar el motor

La siguiente pregunta debe ser.

¿Dónde comienza realmente la simulación?

El objetivo consiste en localizar el equivalente funcional de.

Generator.java

No importa el nombre.

Puede llamarse.

Simulator

Engine

Runner

Generator

Simulation

Lo importante es encontrar el componente que coordina todo el proceso.

14.8 Paso 4. Localizar el paciente

La siguiente búsqueda debe responder.

¿Cuál es el objeto que representa al paciente?

No debe buscarse un archivo llamado Person.

Debe buscarse el objeto que almacena.

- atributos;
- enfermedades;
- historia clínica;
- estado actual.

Ese será el verdadero equivalente de Person.

14.9 Paso 5. Localizar la historia clínica

La siguiente cuestión consiste en responder.

¿Dónde se almacenan los eventos clínicos?

Queremos encontrar el equivalente de.

HealthRecord

Las preguntas principales serán.

- ¿Existe una clase independiente?
 - ¿Cómo se registran los diagnósticos?
 - ¿Cómo se almacenan los procedimientos?
 - ¿Cómo se relacionan las consultas?
-

14.10 Paso 6. Localizar los módulos

A continuación debemos determinar cómo representa Python los protocolos clínicos.

Existen varias posibilidades.

Opción A

Mantener los JSON.

modules/

↓

JSON

Opción B

Convertir cada módulo en una clase.

class HypertensionModule

Opción C

Utilizar funciones.

`simulate_hypertension()`

El objetivo consiste en identificar cuál de estos enfoques se ha adoptado.

14.11 Paso 7. Localizar el mecanismo de ejecución

Ahora debemos responder.

¿Cómo ejecuta Python los módulos?

Las posibilidades son.

- máquina de estados;
- flujo secuencial;
- programación orientada a eventos;
- otro paradigma.

Esta pregunta probablemente sea una de las más importantes de todo el análisis.

14.12 Paso 8. Localizar la exportación

Posteriormente debemos localizar.

- FHIR.
- CSV.
- JSON.

Y responder.

¿La exportación continúa desacoplada del motor?

Si la respuesta es afirmativa.

La filosofía arquitectónica original probablemente se mantiene.

14.13 Qué comparar

Durante el análisis nunca debemos comparar nombres.

Debemos comparar responsabilidades.

Ejemplo.

Incorrecto.

Generator.java

↓

Generator.py

Correcto.

Componente que coordina la simulación

↓

Componente equivalente

Este cambio de perspectiva resulta fundamental.

14.14 Tabla de análisis

Durante el estudio del repositorio conviene mantener una tabla como la siguiente.

Concepto	Jav a	Pytho n	Cambio s
Motor			
Paciente			
Historia clínica			
Módulos			
Estados			

Exportadores

Esta tabla constituirá el núcleo del análisis.

14.15 Preguntas de ingeniería

Cada componente debería analizarse respondiendo siempre las mismas preguntas.

Responsabilidad

¿Qué hace?

Dependencias

¿De quién depende?

Entradas

¿Qué recibe?

Salidas

¿Qué produce?

Equivalencia

¿Existe un componente equivalente en Java?

Cambios

¿Qué diferencias aparecen?

14.16 Qué NO comparar

Resulta poco útil comparar.

- nombres de variables;
- longitud del código;
- estilo de programación.

Lo importante consiste en comparar.

- responsabilidades;
 - arquitectura;
 - flujo de datos;
 - patrones de diseño.
-

14.17 Metodología de documentación

Cada vez que identifiquemos un componente deberíamos completar una ficha.

Ejemplo.

Componente

Patient

Equivalente Java

Person

Responsabilidad

Representar el estado del paciente.

Diferencias

Añade atributos específicos del SNS.

Impacto arquitectónico

Bajo.

La filosofía se mantiene.

Este procedimiento permite documentar rápidamente proyectos de gran tamaño.

14.18 Flujo recomendado

El proceso completo puede resumirse mediante el siguiente diagrama.

README

↓

Arquitectura

↓

Motor

↓

Paciente

↓

Historia clínica



Módulos



Estados



Exportadores



FHIR



Comparación

14.19 Resultado esperado

Al finalizar el análisis deberíamos ser capaces de responder.

- ¿Qué permanece igual?
- ¿Qué cambia?
- ¿Por qué cambia?
- ¿Era necesario dicho cambio?
- ¿La arquitectura mejora o empeora?

Estas preguntas poseen mucho más valor que cualquier comparación línea por línea.

Resumen del capítulo

Al finalizar este capítulo deben haberse comprendido las siguientes ideas fundamentales.

- El análisis de una adaptación debe comenzar por la arquitectura y no por el código.
 - Las responsabilidades son más importantes que los nombres de las clases.
 - Debe mantenerse una metodología sistemática de documentación.
 - El objetivo consiste en comprender decisiones de diseño y no únicamente diferencias de implementación.
 - Comparar arquitecturas resulta mucho más útil que comparar sintaxis.
-

Engineering Notes

Principio fundamental

Nunca compares archivos. Compara responsabilidades.

Regla principal

Si dos componentes realizan la misma función arquitectónica, son equivalentes aunque estén escritos en lenguajes distintos y tengan nombres completamente diferentes.

Idea fundamental del capítulo

La ingeniería inversa consiste en reconstruir las decisiones de diseño de un sistema, no simplemente en leer su código fuente.

 **Comentario del autor (el consejo que creo que más te ayudará en el IRYCIS)**

Pablo, si solo pudiera darte **un consejo** para cuando recibas el repositorio Python sería este:

No abras el primer archivo que encuentres.

Dedica primero **15-20 minutos** a entender cómo está organizado el proyecto. Mira carpetas, nombres, responsabilidades y relaciones.

Ese pequeño esfuerzo inicial te ahorrará horas de confusión.

Y hay una segunda idea que me parece todavía más importante:

No intentes demostrar que sabes Java o Python. Intenta demostrar que entiendes arquitectura.

Porque, por todo lo que me has contado, creo que eso es exactamente lo que el equipo necesita: alguien capaz de explicar **por qué** la adaptación ha cambiado ciertas piezas y **qué impacto tienen esos cambios** en el diseño global del sistema.

CAPÍTULO 15

Framework Profesional para Comparar la Implementación Java y la Adaptación Python

15.1 Introducción

El objetivo de este capítulo consiste en definir una metodología sistemática para comparar la implementación original de Synthea desarrollada en Java con la adaptación desarrollada por el IRYCIS en Python.

Esta comparación no debe realizarse a nivel sintáctico.

Comparar Java con Python línea por línea carece prácticamente de utilidad.

Ambas implementaciones utilizan lenguajes distintos, estilos de programación diferentes y posiblemente estructuras completamente distintas.

El verdadero objetivo consiste en responder una única pregunta.

¿La implementación Python conserva la arquitectura conceptual de Synthea o introduce un nuevo modelo de simulación?

Todo el análisis debe orientarse hacia esa cuestión.

15.2 Principio fundamental

La regla más importante de toda la metodología es la siguiente.

Nunca comparar archivos. Comparar responsabilidades.

Por ejemplo.

Incorrecto.

Generator.java



Generator.py

Correcto.

Componente que coordina la simulación



Componente equivalente

El nombre es irrelevante.

La responsabilidad es lo importante.

15.3 Metodología general

El análisis debe realizarse en cinco fases.

Fase 1

Comprender la arquitectura Python



Fase 2

Encontrar equivalencias



Fase 3

Identificar diferencias



Fase 4

Justificar las diferencias



Fase 5

Evaluar el impacto arquitectónico

15.4 Fase 1

Comprender la arquitectura

Antes de leer código debe responderse.

¿Dónde empieza la simulación?

¿Existe un objeto central?

¿Dónde vive la historia clínica?

¿Cómo se ejecutan los protocolos?

¿Cómo termina la simulación?

Hasta responder estas preguntas no debe estudiarse el código detalladamente.

15.5 Fase 2

Encontrar equivalencias

Construiremos la siguiente tabla.

Responsabilidad	Java	Python
Coordinar simulación	Generator	
Representar paciente	Person	
Historia clínica	HealthRecord	
Protocolo clínico	Module	
Estado	State	
Exportación	Exporter	

Esta tabla constituye el mapa arquitectónico de ambas implementaciones.

15.6 Fase 3

Identificar diferencias

Cada diferencia deberá clasificarse.

Tipo A

Cambio sintáctico.

Ejemplo.

Java.

`person.record`

Python.

`patient.record`

No existe cambio arquitectónico.

Tipo B

Cambio estructural.

Ejemplo.

Una clase desaparece.

Otra asume su responsabilidad.

Tipo C

Cambio arquitectónico.

Ejemplo.

Desaparece la máquina de estados.

El motor cambia completamente.

Este tipo de modificaciones serán las más importantes.

15.7 Fase 4

Justificar los cambios

Toda diferencia debe responder tres preguntas.

¿Qué cambia?

¿Por qué cambia?

¿Era necesario?

Por ejemplo.

Claims

↓

SNS

Cambio.

El sistema económico.

Motivo.

España no utiliza el mismo modelo de aseguradoras.

Impacto.

Arquitectónico medio.

15.8 Fase 5

Evaluar el impacto

Cada modificación puede clasificarse.

Impacto	Significado
Bajo	Cambio superficial
Medio	Cambio funcional
Alto	Cambio arquitectónico

Ejemplo.

Cambiar.

Person

↓

Patient

Impacto.

Bajo.

Cambiar.

State Machine

↓

Programación imperativa

Impacto.

Muy alto.

15.9 Checklist de análisis

Cada componente deberá analizarse siguiendo exactamente las mismas preguntas.

Motor

¿Existe?

¿Cómo comienza?

¿Cómo finaliza?

¿Qué coordina?

Paciente

¿Cómo se representa?

¿Qué información almacena?

¿Existe un único objeto?

Historia clínica

¿Cómo se registra?

¿Cómo se relaciona con el paciente?

Módulos

¿Siguen existiendo?

¿Son JSON?

¿Son clases?

¿Son funciones?

Estados

¿Siguen siendo máquinas de estados?

¿Existe otro paradigma?

Exportación

¿FHIR continúa separado?

¿CSV continúa separado?

15.10 Matriz de comparación

Durante las prácticas recomiendo mantener una tabla como esta.

Componente	Java	Python	Diferencias	Impacto
------------	------	--------	-------------	---------

Generator				
-----------	--	--	--	--

Person				
--------	--	--	--	--

HealthRecord				
--------------	--	--	--	--

Module				
--------	--	--	--	--

State				
-------	--	--	--	--

Exporters				
-----------	--	--	--	--

Costs				
-------	--	--	--	--

Providers				
-----------	--	--	--	--

15.11 Preguntas clave

Durante todo el análisis únicamente deberíamos intentar responder estas diez preguntas.

1

¿Se mantiene la filosofía de simulación?

2

¿Se mantiene el objeto central del paciente?

3

¿Se mantiene la historia clínica independiente?

4

¿Los módulos siguen siendo independientes?

5

¿Continúa existiendo una máquina de estados?

6

¿Se conserva la separación entre simulación y exportación?

7

¿Qué componentes cambian?

8

¿Por qué cambian?

9

¿Las modificaciones responden al sistema sanitario español?

10

¿La arquitectura mejora o empeora?

15.12 Errores que debemos evitar

Durante el análisis existen varios errores muy frecuentes.

Error 1

Comparar únicamente sintaxis.

Error 2

Comparar nombres.

Error 3

Leer archivos sin comprender la arquitectura.

Error 4

Pensar que una implementación distinta implica necesariamente una peor arquitectura.

Error 5

Asumir que Python debe parecerse a Java.

Puede implementar exactamente la misma filosofía mediante una estructura completamente diferente.

15.13 Resultado esperado

Al finalizar el análisis deberíamos ser capaces de escribir un informe como este.

Arquitectura

La adaptación mantiene el modelo general de simulación.

Paciente

Existe un equivalente funcional de Person.

Historia clínica

Se conserva la separación respecto al estado del paciente.

Módulos

Han sido reimplementados en Python.

Motor

Mantiene la filosofía del original.

Cambios específicos

Se introducen modificaciones relacionadas con.

- Atención Primaria.
- SNS.

- Costes.
 - Providers.
-

Conclusión

La adaptación mantiene la filosofía arquitectónica de Synthea incorporando modificaciones necesarias para representar el sistema sanitario español.

15.14 Framework definitivo

Todo el trabajo realizado durante las prácticas puede resumirse mediante el siguiente esquema.

Synthea Java

↓

Comprender arquitectura

↓

Comprender responsabilidades

↓

Comprender patrones

↓

Comprender flujo

↓

Recibir adaptación Python

↓

Localizar equivalencias



Documentar diferencias



Justificar cambios



Evaluar impacto



Conclusiones

Conclusiones finales del libro

Tras el estudio realizado pueden extraerse las siguientes conclusiones.

Synthea no debe entenderse como un proyecto Java, sino como una arquitectura de simulación clínica altamente desacoplada.

El verdadero conocimiento clínico reside en los módulos y no en el motor.

El motor constituye un intérprete de protocolos clínicos implementados mediante máquinas de estados.

La historia clínica representa la consecuencia natural de la simulación y no el punto de partida.

La separación entre Person, HealthRecord, módulos y exportadores constituye la principal fortaleza arquitectónica del proyecto.

Una adaptación nacional debe preservar estas decisiones de diseño siempre que sea posible.



Engineering Notes

La pregunta más importante de todo el proyecto

¿La adaptación Python conserva las decisiones arquitectónicas fundamentales de Synthea o introduce un nuevo paradigma de simulación?

Si al finalizar las prácticas eres capaz de responder con argumentos técnicos a esa pregunta, habrás cumplido exactamente el objetivo que, por lo que me has contado, espera el IRYCIS.